

UNIVERSIDAD DE EXTREMADURA

FACULTAD DE CIENCIAS

GRADO EN MATEMÁTICAS

TRABAJO FIN DE GRADO

OPTIMIZACIÓN DE HORARIOS

MARÍA DOLORES GUTIÉRREZ RANGEL

JULIO 2026

José Antonio Zarandieta Morán, profesor del profesor del Departamento de Ingeniería de Sistemas Informáticos y Telemáticos de la Universidad de Extremadura,

INFORMA:

Que Dña. María Dolores Gutiérrez Rangel ha realizado bajo su dirección el Trabajo Fin de Grado y considera que la memoria reúne los requisitos necesarios para su evaluación.

Badajoz, 1 de julio de 2026

Agradecimientos

Quiero agradecer a mis padres y a mi hermana, quienes han dado tanto o más de sí mismos que yo para que hoy pueda llegar hasta aquí. A Manuel, por acompañarme y apoyarme incondicionalmente. Y a mis amigos, por estar siempre ahí.

También quiero agradecer a mi tutor, José Zarandieta, por su orientación y ayuda durante la realización de este Trabajo Fin de Grado. Asimismo, quiero agradecer a Jacinto Martín, quien desde el primer momento confió en esta idea. La disponibilidad, el apoyo y la dedicación de ambos han sido fundamentales para el desarrollo de este trabajo y para materializar esta idea.

Por último, quiero agradecer a los profesores para los que sus alumnos no hemos sido solo números en una lista, sino personas.

A todos ellos, muchas gracias.

Índice general

Resumen **1**

Introducción **3**

Marco teórico **5**

1. Programación entera **8**
1.1. Definiciones 8
1.2. Métodos exactos de solución 9
1.2.1. Método Gráfico 9
1.2.2. Método de Redondeo 9
1.2.3. Método de Enumeración Completa 9
1.2.4. Método de Bifurcación y Acotación 10
1.3. Complejidad computacional del problema 14
1.4. Métodos heurísticos de resolución 16
1.4.1. Algoritmo genético 16
1.4.2. Enfriamiento simulado 19
1.4.3. Búsqueda tabú 21
1.4.4. Búsqueda local iterada 24

2. Aplicación programación entera a horarios **26**
2.1. Planteamiento del problema 26
2.2. Resolución del problema con $RStudio$ 30

3. Programación de horario **36**
3.1. Introducción 36
3.1.1. Selección de Metodologías y Arquitecturas 37
3.2. Base de datos 37
3.3. Interfaces 43
3.3.1. $index.php$ 45
3.3.2. $index_script.js$ 46
3.3.3. $horario.php$ 46

3.3.4. *horario_script.js* 47
3.3.5. *index_style.css* y *horario_style.css* 47

Conclusión **50**

Bibliografía **52**

Apéndices **54**

Resumen

La planificación de horarios es un problema recurrente de optimización cuya resolución se vuelve considerablemente más compleja al aumentar el número de restricciones que el horario debe cumplir. En este trabajo se estudia la aplicación de la programación entera a la optimización de los horarios de clase en los grados de Matemáticas, Física y Estadística y en el doble grado en Matemáticas y Estadística de la Universidad de Extremadura con el objetivo final de minimizar las coincidencias de los alumnos matriculados en el curso 2024/2025.

Primero, se desarrolla un marco teórico que facilite la comprensión al lector de los métodos utilizados posteriormente. A continuación, se formula el problema principal del trabajo y se plantea su resolución mediante distintos métodos heurísticos. Los resultados de este trabajo arrojan que el enfriamiento simulado es el mejor algoritmo para la resolución de este problema por su relación entre la mejora de la solución y el tiempo necesario para alcanzarla.

Finalmente, se diseña e implementa una base de datos destinada a gestionar y almacenar la información relativa a estudiantes, profesorado y asignaturas, facilitando el tratamiento de datos y la posterior integración de los resultados obtenidos. Además, la base de datos se enlaza a una página web que muestra por alumno de forma automatizada su horario individualizado.

Palabras clave: programación entera, optimización de horarios, heurísticas, base de datos, desarrollo web.

Introducción

La coincidencia de horarios entre distintas asignaturas para los estudiantes es bastante común. Estas coincidencias impiden la asistencia regular a una o varias de ellas, lo cual supone un problema para su seguimiento diario y posterior estudio. Con el objetivo de profundizar en el uso de la programación entera, se van a estudiar los horarios semanales de clases de los grados de Matemáticas, Estadística y Física junto al doble grado en Matemáticas y Estadística de la Universidad de Extremadura. El objetivo es minimizar las coincidencias totales de los alumnos matriculados en el curso escolar 2024/2025 de dichas titulaciones.

Además, como parte de este trabajo, se ha diseñado e implementado una base de datos para almacenar y gestionar de forma eficiente la información recopilada. También se ha desarrollado una página web con el objetivo de que la consulta de datos y resultados sea mucho más sencilla. Esto ha permitido desarrollar en gran medida las competencias en estas áreas, complementando la formación proporcionada por el grado, siendo este uno de los objetivos del Trabajo de Fin de Grado.

Marco teórico

Introducción a la optimización

La optimización es una rama de las matemáticas encargada del estudio de problemas cuyo objetivo es encontrar el mejor valor que alcanza cierta función dentro de un conjunto de restricciones.

Su aplicación se encuentra tanto dentro como fuera del ámbito académico. En Matemáticas, la optimización se utiliza en áreas como teoría de grafos y lógica. Fuera de esta, la encontramos en la gestión de recursos para optimizar la producción, planificación de carreteras, diseño de redes de transporte, entre otras aplicaciones. La optimización constituye una herramienta fundamental para la toma de decisiones en problemas reales de asignación de recursos y planificación (Capítulo 1 [1]).

Un problema de optimización consiste en minimizar (o maximizar) una función $f(x)$, la cual denominaremos función objetivo, sujeta a un conjunto de funciones $g_i(x) \geq 0$ llamadas restricciones.

$$\begin{aligned} \text{mín} \quad & f(x) \\ \text{s.a.} \quad & g_i(x) \geq 0 \end{aligned}$$

Dichas restricciones definen un conjunto de soluciones factibles, S. Por tanto, la optimización tiene como objetivo encontrar un punto $\bar{x} \in S$ tal que $f(\bar{x}) \leq f(x)$ $\forall x \in S$. Al punto $\bar{x}$ se le denomina solución óptima.

Dentro de la optimización pueden distinguirse varias clases de problemas dependiendo de la función objetivo, las restricciones y el dominio de las variables

- Programación lineal: Problemas en los que la función objetivo y las restricciones son lineales. Es la forma más estudiada y utilizada, por ejemplo, en la asignación de recursos o el transporte de mercancías.
- Programación entera: Las variables toman valores enteros. Importante en contextos de toma de decisiones discreta, como planificación de horarios o asignación de tareas.

- Programación mixta: Combinación de variables reales y enteras.
- Programación no lineal: Una, o varias, de las funciones del problema no es lineal.
- Programación convexa: La función objetivo f y el conjunto de soluciones son convexos.
- Programación dinámica: El problema es recursivo.

Para asegurarnos de que el problema de optimización tiene solución, debe verificarse que el conjunto de soluciones sea no vacío y cerrado, mientras que la función objetivo debe ser continua sobre dicho conjunto.

Contexto histórico

Aunque los orígenes de la programación lineal pueden remontarse hasta Fourier, es durante la segunda guerra mundial cuando se plantea como un modelo matemático. El ejército aliado se enfrenta a problemas complejos a la hora de gestionar y planificar los recursos disponibles, desde el transporte de suministros hasta la distribución del personal. Esto impulsó el desarrollo de un modelo matemático capaz de optimizar la toma de decisiones. Es así como nace la investigación operativa, y más concretamente, la programación lineal.

Tras la guerra, muchos de los científicos que participaron en el desarrollo de este modelo vieron las numerosas aplicaciones que podrían hacerse a la industria, lo que provocó un gran avance de este campo. Fue en 1947 cuando George B. Dantzig, físico y matemático consejero de las Fuerzas Armadas de los Estados Unidos, presentó por primera vez un problema de programación lineal y desarrolló el método simplex para su resolución.

Generalmente, se necesita una gran cantidad de operaciones aritméticas para la resolución de uno de estos problemas complejos. No es hasta la década de 1980 que el desarrollo de ordenadores cada vez más potentes permite realizar los cálculos necesarios. Es entonces cuando la programación lineal sufre un crecimiento exponencial al ser más accesible. A partir de aquí, surgen nuevas ramas de investigación como la programación entera.

Actualmente, esta rama de las matemáticas sigue siendo objeto de estudio para enfrentarnos a problemas cada vez más complejos de optimización presentes tanto a nivel industrial como social.

(El marco histórico sigue la información presentada en [3, Cap. 1])

Capítulo 1

Programación entera

1.1. Definiciones

Es necesario iniciar definiendo algunos conceptos importantes (definiciones tomadas de [2, Capítulos I.1. – I.3.]).

Definición (poliedro): Sean $A \in \mathcal{M}_{m\times n}$ y $b \in \mathcal{M}_{m\times 1}$ matrices de números reales conocidos y $x \in \mathbb{R}^n$. Un poliedro es el conjunto de puntos $\mathcal{P} = \{x \in \mathbb{R}^n : Ax \leq b\}$.

Definición (problema de programación lineal y solución factible): Sean $\mathcal{P}$ un poliedro en $\mathbb{R}^n$ y $f : \mathcal{P} \to \mathbb{R}$ una aplicación lineal. Un problema de programación lineal, PPL, consiste en calcular el valor óptimo de f en $\mathcal{P}$. Al área que delimita $\mathcal{P}$ la llamaremos área de soluciones factibles y a cada uno de sus puntos solución factible.

Definición (PPL entera y PPL mixta): Sea $\mathcal{P} \subseteq \mathbb{R}^n$ un poliedro y $f : \mathcal{P} \to \mathbb{R}$ una aplicación lineal. Consideremos un PPL en que queremos calcular el valor óptimo de f en $\mathcal{P}_I = \{x \in \mathcal{P} : x_i \in \mathbb{Z} \quad \forall i \in I\}$ donde I es el conjunto de índices de las variables restringidas a tomar valores enteros.

- Si $I = \{1, \ldots, n\}$, es decir, todas las variables toman valores enteros, estamos frente a un problema de programación lineal entera, PPLE.
- Si $I \subset \{1, \ldots, n\}$, es decir, el problema cuenta con variables enteras y variables reales. Entonces, estamos frente a un problema de programación lineal mixta, PPLM.

Definición (PPL binaria): Un problema de programación lineal binaria es un PPLE en el cual $x_i \in \{0, 1\}$. Normalmente, el uso de variables binarias se destina para resolver problemas de decisiones de tipo «si o no».

1.2. Métodos exactos de solución

Los métodos descritos a lo largo de este apartado se basan en [1, Capítulo 4]

1.2.1. Método Gráfico

El método gráfico es utilizado para resolver problemas de programación lineal de, como máximo, tres variables colocando cada una en uno de los ejes cartesianos del espacio tridimensional.

El método consta de los siguientes pasos:

1. Colocar en cada eje una de las variables
2. Representar cada restricción en el gráfico
3. Definir la región de soluciones
4. Considerar los puntos con coordenadas enteras que encontramos en el límite con menor (o mayor en el caso de problemas de maximización) valor de la función objetivo del área de soluciones factibles.
5. Tomar como solución el punto con menor valor de la función objetivo

1.2.2. Método de Redondeo

Este método redondea la solución del problema de programación lineal al conjunto de valores de las variables que sean números enteros y verifiquen las restricciones pedidas por nuestro problema. Siendo relativamente sencillo, este método tiene el problema de que la solución encontrada no garantiza encontrar el punto óptimo.

1.2.3. Método de Enumeración Completa

Este método evalúa la función objetivo en todos los puntos de coordenadas enteras del área de soluciones factibles, lo cual aumenta considerablemente el número de cálculos. Se toma como solución el punto con mejor valor de la función objetivo. Sin embargo, se garantiza la solución óptima del problema.

1.2.4. Método de Bifurcación y Acotación

El objetivo del método de bifurcación y acotación es encontrar la solución entera óptima mediante la partición sucesiva del problema original en subproblemas más pequeños, descartando aquellas regiones que no pueden contener la solución óptima. La ejecución del método se desarrolla de la siguiente manera:

1. Resolver el problema P de programación lineal sin restricción de soluciones a valores enteros.

2. Si la solución es entera, hemos acabado. Si la solución no es entera, realizamos la primera ramificación.

   Para realizar la ramificación comenzamos tomando una de las variables cuyo valor en la solución obtenida no es entero, X_i, y consideramos los dos números enteros que la comprenden, n_1 y n_2: $n_1 < X_i < n_2$.

   Obtenemos así dos ramas y un problema de programación lineal por cada una. Estos tendrán las mismas restricciones que el problema original P más una adicional: $X_i \leq n_1$ para el problema de la primera rama y $X_i \geq n_2$.

3. Se solucionan cada uno de los problemas anteriores:

   - Si la solución obtenida no es factible, la rama se descarta y no se divide más.
   - Si la solución es entera, esta será una cota superior (problema de minimización) o inferior (problema de maximización) tanto para las soluciones enteras como reales. La rama no se divide más.
   - Si la solución no es entera, se repite el paso 2 siempre que el valor de la función objetivo mejore.

El proceso se repite hasta que no haya más ramificaciones, asegurándonos así de haber encontrado la solución óptima, que será la última cota obtenida. Lo encontramos resumido en Pseudocódigo 1.2.1

El método de bifurcación y acotación es el más utilizado por los programas comerciales de optimización.

Pseudocódigo 1.2.1: Método de bifurcación y acotación

Entrada : PPLE P
Salida : Solución óptima entera x_0

```
1  x_0 = ∅;
2  cota = +∞;
3  x = Resolver(P);
4  if x es entero then
5      x_0 = x;
6      return x_0;
7  while existan ramas por explorar do
8      Seleccionar una rama P_i;
9      x = Resolver(P_i);
10     if x no es factible then
11         Descartar rama;
12     else if x es entero then
13         if f(x) < cota then
14             x_0 = x;
15             cota = f(x);
16         Descartar rama;
17     else if f(x) ≥ cota then
18         Descartar rama;
19     else
20         Seleccionar una variable x_j con valor no entero;
21         Crear la rama P_i ∪ {x_j ≤ ⌊x_j⌋};
22         Crear la rama P_i ∪ {x_j ≥ ⌈x_j⌉};
23 return x_0;
```

Ejemplo 1.2.1. *Definimos el problema P de programación lineal:*

$$\begin{aligned} \text{mín} \quad & f(x_1, x_2) = 13x_1 + 12x_2 \\ \textit{s.a.} \quad & 5x_1 + 6x_2 \geq 120 \\ & 4x_1 + 3x_2 \geq 80 \\ & x_1 \geq 0 \\ & x_2 \geq 0 \end{aligned}$$

Llamamos P' al problema de programación lineal entera definido por la misma

función objetivo y las mismas restricciones. Recordemos que nuestro objetivo final es resolver el problema P'.

Comenzamos resolviendo el problema P mediante el uso del software estadístico $RStudio$:

```
install.packages("lpSolve")
library(lpSolve)
f_obj <-c(13,12)
f_rest <- matrix ( c(5,6,
                     4,3), nrow = 2, byrow = TRUE)
f_sig <- c(">=",
           ">=")
f_b <- c(120,
         80)
lp("min", f_obj, f_rest, f_sig, f_b)
lp("min", f_obj, f_rest, f_sig, f_b)$solution
```

Obtenemos que la solución es el punto $(13{,}333333, 8{,}888889)$. Como no es entera, comenzamos con la primera ramificación:

Rama 1: Problema original P más la restricción $x_1 \leq 13$, $P + x_1 \leq 13$

Rama 2: $P + x_1 \geq 14$

Solucionamos el problema de programación lineal de cada rama:

Rama 1: Solución: $(13, 9{,}333333)$, $f(13, 9{,}333333) = 281$

Rama 2: Solución: $(14, 8{,}333333)$, $f(14, 8{,}333333) = 282$

Ambas soluciones son factibles y no enteras, cada una de las ramas se bifurca.

Rama 1 : $P + x_1 \leq 13$
Solución: $S = (13, 9{,}33333)$, $f(S) = 281$

Rama 1.1 : $P + (x_1 \leq 13) + (x_2 \leq 9)$.
Este problema no tiene soluciones factibles, descartamos esta rama.

Rama 1.2 : $P + (x_1 \leq 13) + (x_2 \geq 10)$
Solución: $S = (12{,}5, 10)$, $f(S) = 282{,}5$

Rama 1.2.1 : $P + (x_1 \leq 13) + (x_2 \geq 10) + (x_1 \leq 12) = P + (x_1 \leq 12) + (x_2 \geq 10)$
Solución: $S = (12, 10{,}66667)$, $f(S) = 284$

Rama 1.2.1.1 : $P + (x_1 \leq 12) + (x_2 \geq 10) + (x_2 \leq 10) =$
$= P + (x_1 \leq 12) + (x_2 = 10)$
Este problema no tiene soluciones factibles, descartamos esta rama.

Rama 1.2.1.2 : $P + (x_1 \leq 12) + (x_2 \geq 10) + (x_2 \geq 11) =$
$= P + (x_1 \leq 12) + (x_2 \geq 11)$
Solución: $S = (11{,}75, 11)$, $f(S) = 284, 75$

Rama 1.2.1.2.1 : $P + (x_1 \leq 11) + (x_2 \geq 11)$
Solución: $S = (11, 12)$, $f(S) = 287$
Tomamos esta solución como la solución provisional del problema a la espera de ver si encontramos otra cuyo valor de la función objetivo sea menor.

Rama 1.2.1.2.2 : $P + (x_1 = 12) + (x_2 \geq 11)$
Solución: $S = (12, 11)$, $f(S) = 288$
Comparamos esta solución con la que hemos denominado solución provisional y vemos que el valor de la función objetivo aumenta. Descartamos esta rama.

Rama 1.2.2 : $P + (x_1 = 13) + (x_2 \geq 10)$
Solución: $S = (13, 10)$, $f(S) = 289$
Comparamos esta solución con la que hemos denominado solución provisional. El valor de la función objetivo aumenta, descartamos esta rama.

Rama 2 : $P + x_1 \geq 14$
Solución: $S = (14, 8{,}33333)$, $f(S) = 282$

Rama 2.1 : $P + (x_1 \geq 14) + (x_2 \leq 8)$
Solución: $S = (14{,}4, 8)$, $f(S) = 283{,}2$

Rama 2.1.1 : $P + (x_1 = 14) + (x_2 \leq 8)$
Este problema no tiene soluciones factibles, descartamos esta rama.

Rama 2.1.2 : $P + (x_1 \geq 15) + (x_2 \leq 8)$
Solución: $S = (15, 7{,}5)$, $f(S) = 285$

Rama 2.1.2.1 : $P + (x_1 \geq 15) + (x_2 \leq 7)$
Solución: $S = (15{,}6, 7)$, $f(S) = 286{,}8$

Rama 2.1.2.1.1 : $P + (x_1 = 15) + (x_2 \leq 7)$
Este problema no tiene soluciones factibles, descartamos esta rama.

Rama 2.1.2.1.2 : $P + (x_1 \geq 16) + (x_2 \leq 7)$
Solución: $S = (16, 6{,}66667)$, $f(S) = 288$
El valor de la función objetivo de esta solución supera al de la solución provisional. Todas las posibles soluciones obtenidas de esta rama tendrán un valor de la función objetivo aún mayor, luego podemos descartarlas y por tanto, también esta rama.

Rama 2.1.2.2 : $P + (x_1 \geq 15) + (x_2 = 8)$
Solución: $S = (15, 8)$, $f(S) = 291$
Comparamos esta solución con la que hemos denominado solución provisional . El valor de la función objetivo aumenta, descartamos esta rama.

Rama 2.2 : $P + (x_1 \geq 14) + (x_2 \geq 9)$
Solución: $S = (14, 9)$, $f(S) = 290$
Comparamos de nuevo esta solución con la solución provisional. El valor de la función objetivo aumenta, descartamos esta rama.

Concluimos que la solución del problema P' es el punto $(11, 12)$.

(Ejemplo tomado de [1, Capítulo 4])

1.3. Complejidad computacional del problema

Los problemas de optimización no solo deben centrarse en encontrar una solución óptima, sino también en el coste computacional necesario para resolverlos. La teoría de la Complejidad Computacional clasifica los problemas de optimización según la cantidad de recursos necesarios para su resolución prestando especial atención al tiempo de ejecución. Aunque podemos clasificar los problemas de diferentes formas, en esta ocasión nos centramos en la clase P y la clase NP.

La clase P está formada por los problemas cuyo tiempo necesario para resolverlos eficientemente aumenta de forma polinómica a medida que aumentan sus variables, es decir, problemas que con n variables presentan una complejidad del orden n, n^2, n^3...

Por el contrario, existen problemas cuya complejidad escala hasta 2^n o $n!$, es decir, estos problemas presentan un aumento considerable de cálculos al incrementar el número de variables, haciendo inviable la resolución por métodos exactos. Estos problemas forman la clase NP, más concretamente, la clase NP se define como el conjunto de problemas cuya solución puede verificarse de manera eficiente aunque esta sea difícil de encontrar.

Actualmente, la relación entre P y NP es uno de los debates abiertos más importantes en este campo pues hasta el momento no se ha demostrado si ambas clases coinciden o no. Para resolver esta cuestión los problemas NP-completos son fundamentales. Estos representan los problemas más complejos de NP.

Proposición 1.1. *Si B es NP-completo y $B \in P$, entonces $P = NP$.*

Este resultado muestra la importancia de los problemas NP-completos, pues bastaría encontrar un algoritmo polinómico para resolver uno de ellos para demostrar que todos los problemas de NP pueden resolverse en tiempo polinómico.

Un ejemplo de un problema NP-completo es el problema SAT (Boolean Satisfiability Problem). Este consiste en determinar si es posible asignar valores de verdadero o falso a un conjunto de variables de forma que verifiquen una serie de condiciones impuestas.

Un resultado fundamental en esta teoría es el teorema de Cook-Levin.

Proposición 1.2 (Cook-Levin)**.** *SAT es NP-completo.*

Este fue el primer problema que se consiguió demostrar que pertenece a la clase NP-completa. Sirve de partida para demostrar que otros problemas también pertenecen a esta clase.

Proposición 1.3. *El coloreado de grafos (Graph Coloring) es un problema NP-completo*

El problema de coloreado de grafos consiste en determinar si existe una asignación de, como máximo, k colores a los vértices de un grafo de forma que dos vértices adyacentes no compartan color. Dos vértices son adyacentes si hay alguna arista que los una.

Este tipo de problemas está relacionado con la optimización de horarios. Consideremos el problema P que definiremos en detalle más adelante en Sección 2.1. Sea $G = (V, E)$ un grafo cualquiera. Asociamos a cada vértice $v \in V$ una asignatura y a cada arista $(u, v) \in E$ una restricción que impide que dos asignaturas sean impartidas de forma simultánea. Consideremos que disponemos de k franjas horarias, asignar una asignatura a una franja es colorear su vértice correspondiente de un color.

Supongamos que existe un algoritmo capaz de resolver el problema de optimización de horarios. En tal caso, también podríamos resolver el problema de coloreado de grafos asociado. Por la Proposición 1.3, este último es un problema NP-completo entonces, los problemas de optimización de horarios presentan una complejidad, al menos, tan elevada como la de un problema NP-completo.

A nivel práctico, identificar que un problema es NP-completo justifica el estudio de métodos alternativos a los algoritmos exactos. En lugar de utilizar estos métodos con costes computacionales elevados, se han desarrollado métodos de resolución para obtener soluciones suficientemente buenas con costes mucho menores, los métodos heurísticos.

(Los conceptos de complejidad computacional utilizados en esta sección se han desarrollado siguiendo principalmente [7, Capítulo 7] y [12]).

1.4. Métodos heurísticos de resolución

(Las heurísticas descritas en este apartado se basan en [5] y [6, Capítulos 2 y 3])

1.4.1. Algoritmo genético

Inspirado en la selección natural, se considera cada posible solución como un individuo dentro de una población. Comenzamos con un conjunto aleatorio de soluciones factibles, la población. Cada posible solución, el individuo, se representa con un vector de coordenadas, el cromosoma, donde cada coordenada es una variable o gen. Definimos una nueva función a partir de la función objetivo y las restricciones que evalúa la calidad de cada individuo.

A continuación se describe cómo funciona el método paso a paso:

1. Evaluación: Se evalúa la calidad de cada individuo

2. Selección: Se escogen dos de los mejores individuos para reproducirse. Aquí la idea es favorecer los mejores individuos sin reducir la diversidad

3. Cruce: Los dos individuos seleccionados intercambian parte de su información genética para generar descendencia. Ejemplo explicativo en Tabla 1.1.

Padres	Hijos
$(1, 0, 1, 2, 1, -2)$	$(1, 0, 1, 0, 2, -2)$
$(0, 0, 3, 0, 2, 2)$	$(0, 0, 3, 2, 1, 2)$

Tabla 1.1: Ejemplo de cruce

4. Mutación: Modificamos aleatoriamente algunos genes para evitar que la población sea demasiado similar y explorar otras regiones de soluciones. Ejemplo explicativo en Tabla 1.2.

$(1, 0, 1, 0, 2, -2)$	$(1, 3, 1, 0, 2, -2)$

Tabla 1.2: Ejemplo de mutación

5. Reemplazo: Se comprueba si estos individuos pertenecen a la población y se evalúan sustituyendo de manera total o parcial la generación anterior.

El proceso continúa hasta que se cumple algún criterio de parada: ausencia de mejoras, número máximo de generaciones (iteraciones)... La solución del problema es el progenitor o descendiente con menor función objetivo.

El procedimiento general del algoritmo genético se resume en Pseudocódigo 1.4.1.

Pseudocódigo 1.4.1: Algoritmo Genético

Entrada : Progenitores iniciales p_1 y p_2
Salida : Mejor solución encontrada x_1

while *no se cumpla el criterio de parada* **do**
  $(h_1, h_2) =$ Cruce(p_1, p_2);
  if h_1 *no es solución factible* **then**
    $h_1 =$ Reparar(h_1);
  if h_2 *no es solución factible* **then**
    $h_2 =$ Reparar(h_2);
  $h_1 =$ Mutacion(h_1);
  $h_2 =$ Mutacion(h_2);
  Evaluar p_1, p_2, h_1 y h_2;
  $(p_1, p_2) =$ Seleccionar_Mejores(p_1, p_2, h_1, h_2);
$x_1 =$ Mejor_Solucion(p_1, p_2);
return x_1;

El algoritmo genético permite explorar simultáneamente varias variables, es fácil de adaptar y funciona bien en problemas con muchos mínimos o máximos locales. Sin embargo, sigue siendo un método con alto coste computacional. Además, hay un riesgo elevado de que las soluciones converjan demasiado pronto y no garantiza alcanzar la solución óptima.

Ejemplo 1.4.1. *Consideremos el siguiente PPLE:*

$$\begin{aligned} \text{mín} \quad & f(x) = 3x_1 + 2x_2 + 3x_3 + 5x_4 + x_5 \\ \textit{s.a.} \quad & 5x_1 + 6x_2 + x_3 - 5x_4 \geq 83 \\ & 4x_1 + 3x_2 - 3x_5 \geq 15 \\ & x_1 \geq 0 \\ & x_2 \geq 0 \\ & x_5 \geq -10 \\ & x_3, x_4 \geq -20 \end{aligned}$$

Calculemos una serie de soluciones de dicho problema y evaluemos la función objetivo en tales puntos:

x	$f(x)$
$(12, 11, 8, 10, 5)$	$f((12, 11, 8, 10, 5)) = 137$
$(6, 12, -2, 3, 9)$	$f((6, 12, -2, 3, 9)) = 60$
$(0, 18, 17, -16, 0)$	$f((0, 18, 17, -16, 0)) = 7$
$(10, 13, -4, 8, -10)$	$f((10, 13, -4, 8, -10)) = 74$
$(0, 22, 6, 1, -1)$	$f((0, 22, 6, 1, -1)) = 66$
$(18, 5, 3, 0, -4)$	$f((18, 5, 3, 0, -4)) = 69$

Vamos a tomar como progenitores las dos soluciones con menor función objetivo: $p_1 = (0, 18, 17, -16, 0)$ *y* $p_2 = (6, 12, -2, 3, 9)$*. Intercambiamos parte de sus genes para obtener su descendencia:* $h_1 = (0, 18, -2, -16, 9)$ *y* $h_2 = (6, 12, 17, 3, 0)$*.*

Ahora tenemos que comprobar que los hijos son solución, en este caso h_1 *no es una solución factible, como el método contempla la mutación de genes, utilizamos esto para obtener una que sí lo sea:* $h_1 = (12, 18, -2, -16, 9)$*.*

Por último, calculamos la función objetivo: $f(h_1) = 155$ *y* $f(h_2) = 108$*. Ninguno de los descendientes consigue mejorar a los progenitores, realizamos otro cruce entre los mismos progenitores.*

Se fija como criterio de parada llegar a 5 iteraciones, la tabla Tabla 1.3 resume los pasos seguidos. La solución que obtenemos del problema tras ejecutar el método del algoritmo genético es $(0, 12, -20, -20, 0)$*.*

I	p_1	$f(p_1)$	p_2	$f(p_2)$	h_1	$f(h_1)$	h_2	$f(h_2)$
1	$(0, 18, 17, -16, 0)$	7	$(6, 12, -2, 3, 9)$	60	$(0, 18, -2, -16, 9)$		$(6, 12, 17, 3, 0)$	108
					$(12, 18, -2, -16, 9)$	155		
2	$(0, 18, 17, -16, 0)$	7	$(6, 12, -2, 3, 9)$	60	$(0, 18, 17, 3, 9)$	111	$(6, 12, -2, -16, 0)$	-44
3	$(0, 18, 17, -16, 0)$	7	$(6, 12, -2, -16, 0)$	-44	$(6, 12, 17, -16, 0)$	13	$(0, 18, -2, -16, 0)$	-50
							$(0, 18, -20, -16, 0)$	-104
4	$(0, 18, -20, -16, 0)$	-104	$(6, 12, -2, -16, 0)$	-44	$(6, 18, -2, -16, 0)$	-32	$(0, 12, -20, -16, 0)$	-116
							$(0, 12, -20, -20, 0)$	-136
5	$(0, 18, -20, -16, 0)$	-104	$(0, 12, -20, -20, 0)$	-136	$(0, 12, -20, -16, 0)$	-116	$(0, 18, -20, -20, 0)$	-124

Tabla 1.3: Resumen Ejemplo 1.4.1

1.4.2. Enfriamiento simulado

El método del enfriamiento simulado se basa en el proceso metalúrgico en el que se calienta un material y después se enfría lentamente para obtener una estructura más fuerte. Se asocia la energía del sistema con el valor de la función objetivo.

Es necesario fijar y definir algunos parámetros al inicio del método:

- $\alpha \in (0, 1)$: determina la velocidad de enfriamiento del algoritmo. Valores bajos provocan una reducción rápida de la temperatura y una menor exploración del espacio de soluciones. Por ello, aunque no haya valores universales determinados, es habitual emplear valores altos comprendidos entre 0.90 y 0.99, siendo $\alpha = 0,95$ y $\alpha = 0,99$ los más frecuentes.
- $r \in [0, 1]$: probabilidad mínima de aceptar una solución peor. Para cada iteración se genera un nuevo valor de r
- T_{max}: la temperatura inicial será la temperatura máxima que alcance el sistema. Tomar una temperatura que no sea lo suficientemente alta puede hacer que el método se estanque sin encontrar una solución suficientemente buena.

Estudiemos el método en un problema de minimización paso a paso:

1. Sea x_0 una solución factible cualquiera de nuestro PPLE. Definimos una segunda solución x_1 que varía ligeramente respecto a la primera.

2. Calculamos $\Delta = f(x_1) - f(x_0)$ siendo f la función objetivo del PPLE.

3. Para $\Delta < 0$, la solución mejora, entonces sustituimos x_0 por x_1 y tomamos una nueva solución x_1.

   Para $\Delta > 0$, la solución empeora. Calculamos la probabilidad de aceptar la solución peor $P = e^{-\Delta/T}$ con $T = T_{max}$, en la primera iteración. Generamos un valor aleatorio para r, si $P > r$ aceptamos la solución peor, sustituimos x_0 por x_1 y consideramos una nueva solución x_1.

4. Bajamos la temperatura $T = \alpha T$

Repetimos el proceso hasta cumplir algún criterio de parada: la temperatura alcanza un valor mínimo, número máximo de iteraciones, la solución no mejora... En Pseudocódigo 1.4.2 encontramos resumido el método de enfriamiento simulado.

Pseudocódigo 1.4.2: Enfriamiento Simulado

Entrada : Solución factible inicial x_0, temperatura inicial $T_{\text{máx}}$, factor de enfriamiento α
Salida : Solución encontrada x_1

```
x_1 = x_0;
T = T_máx;
while no se cumpla el criterio de parada do
    x'_0 = Generar_Vecino(x_0);
    Δ = f(x'_0) − f(x_0);
    if Δ < 0 then
        x_0 = x'_0;
    else
        Generar un valor aleatorio r ∈ [0, 1];
        if r < e^{−Δ/T} then
            x_0 = x'_0;
    T = αT;
x_1 = x_0;
return x_1;
```

Este método supuso un avance innovador pues por primera vez se permite aceptar temporalmente una solución peor para escapar de mínimos o máximos locales.

Ejemplo 1.4.2. *Sea P el problema definido en Ejemplo 1.4.1*

Fijemos el parámetro $\alpha = 0,95$ y el valor inicial de $T = 100$. Vamos a tomar una solución factible $x_0 = (12, 11, 8, 10, 5)$ y a partir de esta obtenemos $x_1 = (12, 11, 8, 2, 5)$. Entonces $\Delta = f(x_1) - f(x_0) = 97 - 134 = -40 < 0$. La solución mejora. Por tanto, $x_0 = (12, 11, 8, 2, 5)$. El proceso seguido queda resumido en Tabla 1.4

La solución encontrada para el PPLE es $(5, 15, -7, -22, -10)$. En ese ejemplo, podemos ver la importancia de este método. Fijémonos en la iteración 6, inicia

considerando como solución $(5, 11, -7, -12, -10)$ *cuyo valor de la función objetivo es* -54 *y tras completar la iteración tenemos como solución* $(5, 15, -7, -12, -10)$ *con* $f((5, 15, -7, -12, -10)) = -36$*, es decir, una solución peor. Sin embargo, eso nos permite llegar a una solución con menor valor de la función objetivo.*

I	x_0	$f(x_0)$	x_1	$f(x_1)$	Δ	r	P	T
1	$(12, 11, 8, 10, 5)$	137	$(12, 11, 8, 2, 5)$	97	$-40 < 0$	**X**	**X**	95
2	$(12, 11, 8, 2, 5)$	97	$(12, 11, -7, 2, 5)$	52	$-45 < 0$	**X**	**X**	90,25
3	$(12, 11, -7, 2, 5)$	52	$(12, 11, -7, -12, 5)$	-18	$-70 < 0$	**X**	**X**	85,74
4	$(12, 11, -7, -12, 5)$	-18	$(5, 11, -7, -12, 5)$	-39	$-21 < 0$	**X**	**X**	81,45
5	$(5, 11, -7, -12, 5)$	-39	$(5, 11, -7, -12, -10)$	-54	$-15 < 0$	**X**	**X**	77,38
6	$(5, 11, -7, -12, -10)$	-54	$(5, 15, -7, -12, -10)$	-46	$8 > 0$	0,64	$0{,}902 > 0{,}64$	73,51
7	$(5, 15, -7, -12, -10)$	-46	$(5, 20, -7, -12, -10)$	-36	$10 > 0$	0,88	$0{,}867 < 0{,}88$	69,83
8	$(5, 15, -7, -12, -10)$	-46	$(5, 15, -7, -22, -10)$	-86	$-40 < 0$	**X**	**X**	66,34

Tabla 1.4: Resumen del paso a paso de Ejemplo 1.4.2

1.4.3. Búsqueda tabú

El método de búsqueda tabú toma como base la búsqueda local y incorpora mecanismos de memoria a corto plazo para evitar bucles y estancamiento en mínimos locales.

Definimos la lista tabú, un vector de longitud fija r que almacenará las últimas r variaciones realizadas.

Sea x_0 una solución factible del problema. Consideramos una serie de soluciones factibles creadas a partir del x_0 mediante pequeñas modificaciones (modificar el valor de una variable, intercambiar los valores de dos variables...) que vamos a denominar vecinos de x_0.

Entre ellas, se escoge la que tenga un valor de función objetivo menor y se define como x_1. La variación opuesta por la que hemos obtenido x_1 se guarda en el vector de variaciones, es decir, la variación necesaria para obtener x_0 desde x_1. En Tabla 1.5 encontramos un ejemplo de las variaciones y los elementos guardados en la lista tabú.

Vecino de $x_0 = (0, 0, 2, 3, 5)$	Variación	Elemento lista tabú
$(0, 0, 5, 3, 2)$	Intercambiar los valores de las variables 3 y 5	Intercambiar los valores de las variables 3 y 5
$(0, 0, 2, 3, 4)$	Sumar $(0, 0, 0, 0, -1)$	Sumar $(0, 0, 0, 0, 1)$
$(2, 0, 2, 3, 5)$	Sumar $(2, 0, 0, 0, 0)$	Sumar $(-2, 0, 0, 0, 0)$

Tabla 1.5: Ejemplo de vecinos, variaciones y posibles elementos tabú

Buscamos vecinos de x_1. Se descarta la solución obtenida mediante la variación ya utilizada. Entre el resto de soluciones fijamos como x_2 la solución con menor valor de función objetivo y se guarda el elemento correspondiente en la lista tabú.

En cada iteración descartamos las soluciones obtenidas mediante todas las variaciones guardadas en la lista tabú. Cuando ya está completa, la siguiente variación sustituye a la más antigua de manera que la longitud de la lista siempre sea la misma.

El método termina al cumplir algún criterio de parada. El funcionamiento del método de la búsqueda tabú se resume en Pseudocódigo 1.4.3.

Pseudocódigo 1.4.3: Búsqueda Tabú

Entrada : Solución factible inicial x_0, longitud de la lista tabú r
Salida : Solución encontrada x_1

```
1  x_1 = x_0;
2  tabú = (0, ..., 0);
3  while no se cumpla el criterio de parada do
4      V = Generar_Vecinos(x_0);
5      V = Descartar_Tabu(V, tabú);
6      x'_0 = Mejor_Vecino(V);
7      v = Variacion(x_0, x'_0);
8      x_0 = x'_0;
9      for i = 1 to r - 1 do
10         tabú[i] = tabú[i + 1];
11     tabú[r] = Variacion_Opuesta(v);
12 x_1 = x_0;
13 return x_1;
```

Ejemplo 1.4.3. *Se considera de nuevo el problema definido en Ejemplo 1.4.1 .*

Fijemos la longitud del vector tabú en 3. Sea $x_0 = (12, 11, 8, 10, 5)$ *una solución factible del problema. Calculamos algunos vecinos de* x_0 *que sean solución*

x	$f(x)$
$(14, 11, 8, 10, 5)$	143
$(12, 11, 8, 4, 5)$	107
$(12, 18, 8, 10, 5)$	151
$(12, 11, 10, 8, 5)$	133

Definimos $x_1 = (12, 11, 8, 4, 5)$ *obtenida sumando* -6 *a la cuarta variable, luego* $tabu = ((0, 0, 0, 6, 0), 0, 0)$*. Calculamos los vecinos de* x_1*:*

x	$f(x)$
$(11, 12, 8, 4, 5)$	106
$(12, 6, 8, 4, 5)$	97
$(12, 11, 8, 4, 8)$	110
$(12, 11, 8, 5, 4)$	111
$(9, 11, 8, 4, 5)$	98

El mejor vecino es $(12, 6, 8, 4, 5)$*, tomamos esta solución como* x_2*. Entonces:*

$$tabu = ((0, 0, 0, 6, 0),\ (0, 5, 0, 0, 0),\ 0)$$

Vecinos de x_2*:*

x	$f(x)$
$(12, 8, 6, 4, 5)$	95
$(12, 6, 12, 4, 5)$	109
$(12, 6, 8, 2, 5)$	87
$(12, 6, 8, 4, -10)$	82

Tomamos $x_3 = (12, 6, 8, 4, -10)$*, tenemos:*

$$tabu = ((0, 0, 0, 6, 0),\ (0, 5, 0, 0, 0),\ (0, 0, 0, 0, 15))$$

Vecinos de x_3*:*

x	$f(x)$
$(12, 6, 8, -10, 4)$	26
$(12, 8, 6, 4, -10)$	80
$(12, 6, 8, -6, -10)$	32
$(12, 6, 12, 4, -10)$	94

Así, $x_4 = (12, 6, 8, -10, 4)$*. Ahora el vector tabú ya está completo, por lo que esta variación sustituye a la más antigua. Para asegurarnos de siempre eliminar la variación más vieja vamos a desplazar los elementos del vector de manera que el primer elemento sea el siguiente a eliminar*

$$tabu = ((0, 5, 0, 0, 0),\ (0, 0, 0, 0, 15),\ Intercambio(4, 5))$$

Vecinos de x_4:

x	$f(x)$
$(12, 6, 8, -4, 4)$	56
$(12, 15, 8, -10, 4)$	44
$(12, 6, 8, -17, 4)$	-9
$(12, 6, 8, -10, -10)$	12

Finalizamos con $x_5 = (12, 6, 8, -17, 4)$ como solución del problema y $tabu = ((0, 0, 0, 0, 15), intercambio(4, 5), (0, 0, 0, -7, 0))$

1.4.4. Búsqueda local iterada

La búsqueda local iterada se basa, tal y como indica su nombre, en realizar de manera repetida una búsqueda local.

Pasos del método:

1. Solución inicial: Tomamos una solución factible del PPLE.

2. Búsqueda local: Exploramos las soluciones vecinas, se toma una cuyo valor de la función objetivo mejora. Calculamos los vecinos de esta nueva solución y repetimos el proceso. Paramos al encontrar una solución que no tenga vecinos con un menor valor de la función objetivo, es decir, cuando encontremos un mínimo local.

3. Variación de la solución: Modificamos la solución obtenida. La variación que realicemos debe ser lo suficientemente grande para que salga de la región del mínimo local, pero no lo suficiente para perder la información que ya hemos obtenido.

Se repite hasta cumplir algún criterio de parada.

El método de la búsqueda local iterada se resume en el Pseudocódigo 1.4.4.

Pseudocódigo 1.4.4: Búsqueda Local Iterada

Entrada : Solución factible inicial x_0
Salida : Mejor solución encontrada x_1

```
1  x_1 = x_0 ;
2  while no se cumpla el criterio de parada do
3      repeat
4          V = Generar_Vecinos(x_0);
5          x'_0 = Mejor_Vecino(V);
6          if f(x'_0) < f(x_0) then
7              x_0 = x'_0;
8          else
9              break;
10     until se alcance un mínimo local;
11     if f(x_0) < f(x_1) then
12         x_1 = x_0;
13     x_0 = Perturbar(x_0);
14 return x_1;
```

Capítulo 2

Aplicación programación entera a horarios

2.1. Planteamiento del problema

El objetivo del problema es modificar los días y las horas en los que se imparte cada asignatura para que el número de coincidencias totales entre los alumnos disminuya.

Para la realización del problema se han tomado las siguientes consideraciones:

- Se ha recogido, para una muestra de estudiantes, el listado de asignaturas en las que están matriculados en el curso 2024/2025.
- Las asignaturas se separan por semestres (1-8 para los grados y 1-10 para el doble grado).
- Las asignaturas con el mismo profesorado no pueden ser simultáneas.
- No hemos permitido que asignaturas del mismo cuatrimestre y grado coincidan en horario. Esto nos ha llevado a ciertos inconvenientes con las asignaturas del Doble Grado en Matemáticas y Estadística que se imparten de manera conjunta al Grado en Estadística, pero siendo de distintos cursos para cada titulación. Algunas asignaturas que forman parte del quinto y último curso del doble grado se cursan durante el cuarto año para el caso de los estudiantes de estadística. Para que la restricción pudiese aplicarse para ambas titulaciones sin que afectara a asignaturas no comunes, hemos considerado que se trata en cada caso de asignaturas distintas que deben compartir horario. Para que la restricción anterior, referente al profesorado, no cause conflicto con esta modificación, estas asignaturas dobles no cuentan con profesorado registrado.

Esta situación no la encontramos con las asignaturas comunes con el Grado en Matemáticas pues se imparten en el mismo cuatrimestre para ambas titulaciones, lo cual nos permite obviar esta distinción.

- Cada asignatura se imparte durante 4 horas semanales y con duración máxima de dos horas diarias.
- Franjas horarias fijas, todas las clases empiezan y terminan a la vez.
- Las asignaturas comunes en varios grados deben impartirse conjuntamente.
- Se ha realizado una distinción entre las asignaturas con el mismo nombre pero que no se cursan simultáneamente como puede ser el caso de Cálculo I. Esta asignatura se imparte en las cuatro titulaciones que estamos considerando en este estudio. Sin embargo, en tres de ellas se hace de forma conjunta, mientras que para el Grado en Física se da de forma independiente. Por tanto, el horario en ambos casos no necesita coincidir, por lo que las hemos considerado como asignaturas distintas.
- Respecto a las asignaturas optativas no se ha impuesto ningún tipo de restricción. Estas pueden coincidir con cualquier otra asignatura de cualquier curso. Solo se ha mantenido el semestre en el que actualmente se imparte según el plan de estudios de cada una de ellas.
- Se han obviado las asignaturas cuyas horas lectivas se imparten en su totalidad en horario de tarde y aquellas que solo se imparten algunas semanas en horario de mañana.

Teniendo esto en cuenta, planteamos el problema. Comenzamos definiendo una serie de conjuntos que serán utilizados:

- A: Conjunto de asignaturas.
- E: Conjunto de estudiantes.
- P: Conjunto de profesores.
- D: Conjunto de días lectivos por semana.
- H: Conjunto de franjas horarias disponibles.
- G: Conjunto de grados.
- C: Conjunto de cuatrimestres.

Definamos la función objetivo de nuestro problema. Queremos minimizar el número total de coincidencias de clases entre los estudiantes de los grados del estudio. Definimos una variable x que va a controlar si cierta asignatura se imparte en un momento determinado: $x_{a,d,h} = 1$ si la asignatura a se imparte el día d en la franja horaria h y $x_{a,d,h} = 0$ en caso contrario. Por tanto, nos encontramos frente a

un problema de programación lineal binaria.

Una coincidencia se da cuando el estudiante tiene más de una asignatura en una misma franja horaria para un determinado día, es decir, cuando

$$\sum_{a \in A_e} x_{a,d,h} \geq 2$$

Luego, el número de coincidencias viene dado por:

$$\text{máx}(0, \sum_{a \in A_e} x_{a,d,h} - 1)$$

Queremos minimizar la suma total de estas coincidencias para todos los estudiantes, días y franjas, por tanto, la función que deseamos minimizar es la siguiente:

$$g(x) = \sum_{e \in E} \sum_{d \in D} \sum_{h \in H} \text{máx}(0, \sum_{a \in A_e} x_{a,d,h} - 1)$$

Sin embargo, esta función no es lineal y no puede ser optimizada utilizando los métodos de programación lineal explicados a lo largo de este trabajo, luego es necesario transformarla en una función lineal equivalente. Para ello definimos una variable auxiliar $o_{e,d,h}$ tal que $o_{e,d,h} \geq 0$ y $o_{e,d,h} \geq \sum_{a \in A_e} x_{a,d,h} - 1$ de manera que la función objetivo queda de la forma:

$$f(x, o) = \sum_{e \in E} \sum_{d \in D} \sum_{h \in H} o[e, d, h]$$

Comprobemos que ambas expresiones son equivalentes. Sea

$$z = \sum_{a \in A_e} x_{a,d,h} - 1$$

tal y como ha sido definida la variable auxiliar tiene que ser mayor o igual que 0 y mayor o igual que z.

Si $z \leq 0$, entonces $o_{e,d,h} = 0$. Si por el contrario $z > 0$, entonces necesariamente $o_{e,d,h} \geq z$. Esto equivale a $o_{e,d,h} = \text{máx}(0, z)$. Sustituyendo nuevamente el valor de z, obtenemos:

$$o_{e,d,h} = \text{máx} \left(0, \sum_{a \in A_e} x_{a,d,h} - 1 \right)$$

y, por tanto, ambas expresiones son equivalentes.

El siguiente paso es fijar cada una de las restricciones del problema:

- Cada asignatura se imparte un total de 4 horas por semana:

$$\sum_{d\in D}\sum_{h\in H} x_{a,d,h} = 4 \quad \forall a \in A$$

- Cada asignatura se imparte como máximo dos veces al día:

$$\sum_{h\in H} x_{a,d,h} \leq 2 \quad \forall a \in A\ \forall d \in D$$

- Un profesor no puede dar dos asignaturas simultáneamente:

$$\sum_{a\in A_p} x_{a,d,h} \leq 1 \quad \forall p \in P\ \forall d \in D\ \forall h \in H$$

- No puede haber solapamiento entre asignaturas obligatorias del mismo cuatrimestre y grado:

$$\sum_{\substack{a\in A_g \\ C_a=c}} x_{a,d,h} \leq 1 \quad \forall g \in G\ \forall d \in D\ \forall h \in H\ \forall c \in C$$

Concluimos que el problema P que queremos optimizar es el siguiente:

$$\begin{aligned}
\text{mín} \quad & f(x,o) = \sum_{e\in E}\sum_{d\in D}\sum_{h\in H} o_{e,d,h} \\
\text{s.a.} \quad & o_{e,d,h} \geq 0 \quad \forall e \in E,\ \forall d \in D,\ \forall h \in H \\
& o_{e,d,h} \geq \sum_{a\in A_e} x_{a,d,h} - 1 \quad \forall e \in E,\ \forall d \in D,\ \forall h \in H \\
& \sum_{d\in D}\sum_{h\in H} x_{a,d,h} = 4 \quad \forall a \in A \\
& \sum_{h\in H} x_{a,d,h} \leq 2 \quad \forall a \in A,\ \forall d \in D \\
& \sum_{a\in A_p} x_{a,d,h} \leq 1 \quad \forall p \in P,\ \forall d \in D,\ \forall h \in H \\
& \sum_{\substack{a\in A_g \\ C_a=c}} x_{a,d,h} \leq 1 \quad \forall g \in G,\ \forall d \in D,\ \forall h \in H,\ \forall c \in C \\
& x_{a,d,h} \in \{0,1\} \quad \forall a \in A,\ \forall d \in D,\ \forall h \in H
\end{aligned}$$

2.2. Resolución del problema con $RStudio$

Los problemas de optimización de horarios son problemas NP-completos, lo que justifica el uso de métodos heurísticos. Aun así, en primer lugar se intentó resolver el problema P utilizando un método exacto, más concretamente el paquete $GLPK$ disponible en $RStudio$ que utiliza principalmente el método de bifurcación y acotación. Sin embargo, tras un tiempo bastante elevado el método no conseguía obtener una solución tal y como cabía esperar al ser un problema NP-completo.

Así, se ha recurrido a los métodos heurísticos de solución vistos previamente en el primer capítulo de este mismo trabajo: algoritmo genético, enfriamiento simulado y búsqueda tabú. Con el objetivo de valorar, en este caso, cuál de los métodos funciona mejor, apliquemos cada uno de ellos sobre el problema P estudiando el valor de la función objetivo en la solución alcanzada y el tiempo necesario para obtenerla. Aunque estos métodos cuentan con un paquete en $RStudio$ que implementa cada uno ([8], [9] y [10]), debido a la forma escogida para el tratamiento de datos, se ha optado por realizar el método de forma manual y personalizada.

R es un software estadístico cuyo uso está ampliamente extendido. La facilidad para encontrar información, ejemplos y documentación constituye el primer motivo por el que ha sido elegido para este trabajo. El segundo viene dado por su presencia dentro del Grado en Matemáticas, donde su utilización comienza a introducirse en el primer curso de la titulación mediante la interfaz $RCommander$. Esto facilita la comprensión y el manejo de R de manera que resulta razonable que la herramienta finalmente escogida para el desarrollo de este trabajo sea uno de sus entornos de desarrollo, en este caso, $RStudio$.

Por otro lado, con el fin de simplificar el código y reducir el coste computacional se ha decidido dividir el problema entre las asignaturas impartidas en el primer semestre, P_1, y las del segundo, P_2. Para ello, hemos creado una serie de ficheros de extensión $.csv$ de los cuales contamos con una versión para cada semestre que contiene los datos necesarios para este problema:

- $asig_cuatri.csv$: Indica cada asignatura junto al cuatrimestre en el que se imparte.
- $est_asig.csv$: Relaciona cada estudiante con cada asignatura en la que está matriculado.
- $prof_asig.csv$: Análogo al anterior con los profesores y sus asignaturas.
- $grado_asig.csv$: Relación para cada grado de sus asignaturas obligatorias.

y un archivo común que recoge las asignaturas que deben impartirse de forma

simultánea: $asig_dobles.csv$. Cabe mencionar que todos ellos se crean de forma dinámica tras la implementación de la base de datos y la página web (elementos descritos en el Capítulo 3).

A partir de estos datos se ejecutan los distintos algoritmos implementados en $RStudio$. En la Figura 2.1 encontramos un diagrama de secuencia que muestra el flujo de interacción seguido desde la gestión inicial de los datos hasta el almacenamiento del nuevo horario optimizado creado por el método.

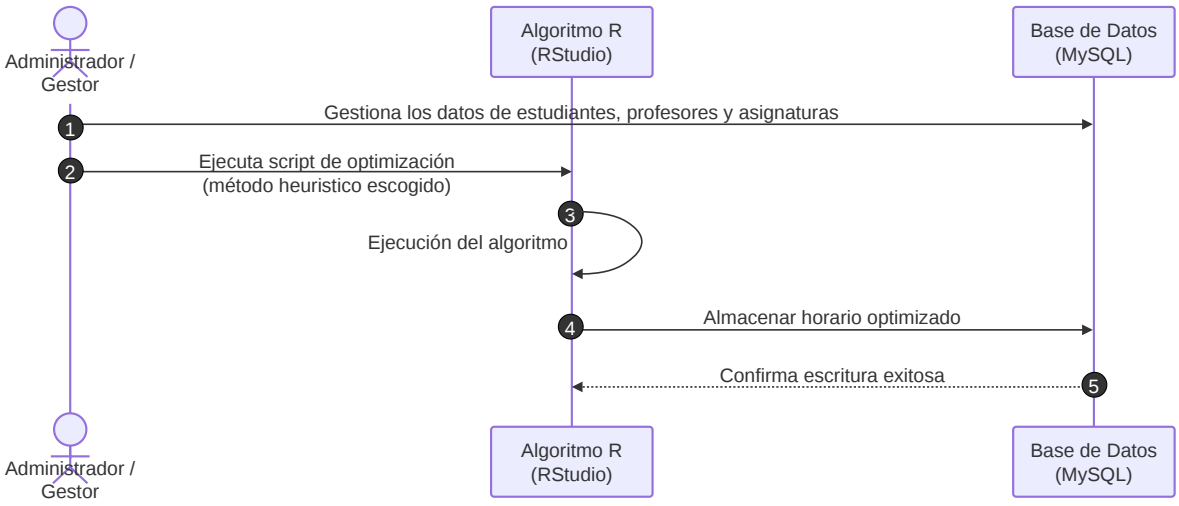


Figura 2.1: Diagrama de secuencia del proceso de optimización

Es importante mencionar que hemos considerado como solución inicial la asignación de horarios actual de los alumnos en lugar de una solución aleatoria para obtener una solución mejor a la que ya tenemos. Puesto que el horario real no cumple todas las condiciones que hemos exigido en nuestro problema, se ha modificado ligeramente añadiendo o quitando horas cuando ha sido necesario. Veamos cómo se han aplicado cada método en P_1 y, equivalentemente, en P_2.

- Algoritmo genético: Cada individuo de la población representa un horario completo del cuatrimestre. Los genes corresponden con las asignaturas y el valor asociado a cada uno de ellos indica la franja horaria asignada a dicha asignatura.

  El primer individuo coincide con la solución inicial. Tras generarse por mutación algunas soluciones factibles más, tomamos los dos mejores individuos como progenitores. La función de cruce de progenitores genera la descendencia mientras que la función de mutación modifica aleatoriamente hijos para evitar la convergencia. Si alguno de los hijos mejora a algún padre, este lo sustituye como progenitor para la nueva generación.

Se ha considerado una población inicial de 30 individuos, 450 generaciones, 4 descendientes por generación y una probabilidad de mutación del $10\,\%$.

(Código completo en Apéndice B)

- Enfriamiento simulado: A partir de la solución inicial se genera una solución vecina y se evalúa en la función objetivo. Si es mejor se acepta directamente. En caso contrario, puede ser aceptado con una probabilidad aleatoria. El proceso se repite varias veces antes de bajar la temperatura.

  Se utilizan los siguientes parámetros: temperatura inicial de 100 y temperatura final de 1. La temperatura se reduce multiplicando por $\alpha = 0,90$ y realizamos 200 iteraciones antes de cada reducción de temperatura.

  (Código completo en Apéndice C)

- Búsqueda tabú: Tras generar varios vecinos de la solución inicial estos son evaluados y se toma el mejor de ellos. Hemos tomado una versión simplificada del método descrito en Sección 1.4. En lugar de almacenar los movimientos hemos optado por almacenar las soluciones para simplificar el código.

  Se realizan 450 iteraciones, se almacenan 25 elementos en la lista tabú de forma simultánea y en cada generación se generan 20 vecinos.

  (Código completo en Apéndice D)

Los parámetros en cada método se determinaron de forma experimental. Como punto de partida se tomaron 300 iteraciones y se realizaron ajustes para asegurar la mejora de la solución y que todos los métodos tuvieran un tiempo de ejecución similar.

Las funciones comunes en los tres métodos han sido definidas en el archivo $datos_comunes.R$ para optimizar los códigos en la medida de lo posible. Aquí encontramos las funciones encargadas de cargar y preparar los datos, evaluar la función objetivo en una solución, generar vecinos, comprobar si una solución es factible y guardar la solución final. La función más importante para la exploración de soluciones es la que genera vecinos. Aquí un vecino se obtiene seleccionando entre dos y cinco asignaturas y modificando una de sus cuatro franjas horarias semanales.

(Código completo en Apéndice A)

Para determinar qué método heurístico arroja mejores soluciones para el problema de optimización de horarios planteado vamos a realizar un análisis estadístico de los resultados obtenidos por cada método. Cada uno de ellos ha sido ejecutado 45 veces de manera independiente y se han recogido en cada ejecución la diferencia entre el valor de la función objetivo de la solución inicial y la final, y el tiempo

empleado. Como queremos aplicar la misma heurística para cada cuatrimestre vamos a considerar la diferencia total (dif_total).

Comenzamos calculando la media y la desviación típica de las variables del estudio agrupando por método (véase Tabla 2.1).

Método	Media dif_total	Desviación típica dif_total	Media $tiempo$	Desviación típica $tiempo$
Algoritmo genético	59,89	13,91	2,11	0,02
Enfriamiento simulado	183,38	11,69	2,19	0,13
Búsqueda tabú	168,47	6,87	2,11	0,02

Tabla 2.1: Tabla resumen de los estadísticos descriptivos estudiados de las variables dif_total y $tiempo$

Aquí podemos observar que el algoritmo genético obtiene soluciones mucho peores que los otros dos métodos. El enfriamiento simulado presenta una mayor diferencia total en el valor de la función objetivo seguido de la búsqueda tabú que se queda algo por debajo.

Para determinar si estas diferencias resultan significativas realizamos un análisis de la varianza. El test de Shapiro-Wilk nos dice que la variable dif_total sigue una distribución normal en los tres métodos. Sin embargo, no podemos asumir varianzas iguales por lo que empleamos el ANOVA de Welch. Los resultados obtenidos muestran que hay diferencias significativas en todos los pares de métodos considerados.

Actuando de forma completamente análoga, estudiamos la variable $tiempo$. En este caso obtenemos que no hay diferencia significativa entre búsqueda tabú y algoritmo genético, siendo este inferior al de enfriamiento simulado.

Por ello, el enfriamiento simulado se presenta como la heurística más adecuada entre las estudiadas, ya que, aunque requiere un tiempo de ejecución ligeramente superior, de tan solo unos segundos, obtiene una mejora notable y estadísticamente significativa en el valor de la función objetivo.

(Análisis estadístico completo en Apéndice I)

Tras obtener el nuevo horario final optimizado veamos algún ejemplo de mejora. Seleccionamos un estudiante, en este caso del grado en Matemáticas. Observemos Figura 2.2, el alumno en cuestión tiene cinco coincidencias en el horario vigente durante el curso 2024/2025. Sin embargo, tras la optimización del problema planteado el número de coincidencias se reduce a dos, véase Figura 2.3.

(Los horarios completos de cada titulación pueden consultarse en Apéndice E, Apéndice F, Apéndice G y Apéndice H).

	Lunes	Martes	Miércoles	Jueves	Viernes
8:15 - 9:05		Métodos Numéricos II			Métodos Numéricos II
9:15 - 10:05		Métodos Numéricos II		Teoría de la Medida	Métodos Numéricos II
10:15 - 11:05	Ampliación de Topología --------- Teoría de la Medida	Ampliación de Topología --------- Teoría de la Medida	Ampliación de Topología --------- Teoría de la Medida	Geometría Diferencial I	Ampliación de Topología
11:15 - 12:05	Geometría Diferencial I	Geometría Diferencial I --------- Estadística Matemática	Estadística Matemática	Estadística Matemática	Geometría Diferencial I --------- Estadística Matemática
12:15 - 13:05					
13:15 - 14:05		Análisis Funcional	Análisis Funcional	Análisis Funcional	
14:15 - 15:05			Análisis Funcional	Análisis Funcional	

Figura 2.2: Horario del alumno durante el primer cuatrimestre del curso 2024/2025

	Lunes	Martes	Miércoles	Jueves	Viernes
8:15 - 9:05		Ampliación de Topología	Teoría de la Medida		Geometría Diferencial I
9:15 - 10:05		Métodos Numéricos II	Teoría de la Medida		Análisis Funcional
10:15 - 11:05	Análisis Funcional	Métodos Numéricos II	Análisis Funcional		Estadística Matemática
11:15 - 12:05	Ampliación de Topología		Geometría Diferencial I	Estadística Matemática	Estadística Matemática
12:15 - 13:05		Teoría de la Medida	Geometría Diferencial I		Análisis Funcional
13:15 - 14:05		Teoría de la Medida	Estadística Matemática	Ampliación de Topología	
14:15 - 15:05	Geometría Diferencial I ------- Métodos Numéricos II		Ampliación de Topología	Métodos Numéricos II	

Figura 2.3: Horario obtenido tras la optimización para el alumno

Capítulo 3

Programación de horario

3.1. Introducción

Con la intención de que cada alumno obtenga el horario semanal de manera automática y personalizada se ha desarrollado una página web que simula el Portal de Servicios que está disponible para cada estudiante de la Universidad de Extremadura añadiendo una nueva función en la cual se pueda encontrar su horario semanal individualizado.

Los datos recogidos para la realización de este trabajo y los obtenidos tras la realización de los problemas que hemos resuelto en el capítulo anterior se han almacenado en una base de datos creada mediante $WAMPServer$ ($Windows$, $Apache$, $MySQL$ y PHP). Este es un entorno de desarrollo con el que podemos ejecutar aplicaciones web de forma local.

Además, se han diseñado dos interfaces mediante el uso de $HTML$ para el diseño de la estructura, CSS para el formato visual y PHP para realizar la conexión entre la base de datos y la lógica del servidor. También se han realizado elementos con $JavaScript$ que permite la interactividad de la página.

Vamos a ir explicando de forma algo más detallada cada uno de los elementos creados en las siguientes secciones de este capítulo.

Por último, mencionar que los datos utilizados para la elaboración de este trabajo han sido recogidos mediante un cuestionario rellenado de forma voluntaria por más de 100 estudiantes de las titulaciones escogidas donde podían identificarse o continuar de forma anónima, opción escogida por gran parte de los participantes.

3.1.1. Selección de Metodologías y Arquitecturas

La implementación del sistema desarrollado a lo largo de este trabajo ha exigido seleccionar distintas herramientas siguiendo dos criterios claros: adecuación al problema y facilidad de uso.

En primer lugar, se ha optado por una arquitectura web de cliente-servidor. Esta permite separar la gestión de los datos y la ejecución de procesos de la interfaz diseñada para el usuario. Así, se facilita el mantenimiento y la actualización de datos.

El siguiente paso consiste en seleccionar un sistema gestor de bases de datos que nos permita almacenar y gestionar la información que recogen las tablas de datos. Se han valorado distintas opciones para almacenar la información en archivos planos, como CSV y $JSON$. Sin embargo, estos han sido descartados por las relaciones entre los datos y la complejidad de realizar consultas y evitar redundancias.

También se ha descartado el uso de bases de datos $NoSQL$. Aunque estas facilitan la gestión de grandes cantidades de datos, la estructura estricta que exige el caso planteado dificulta considerablemente su uso.

Así, se opta finalmente por un Sistema Gestor de Bases de Datos Relacional (SGBDR), más concretamente $MySQL$. Este sistema permite el uso de claves primarias, restricciones de integridad relacional y consultas SQL eficientes [11].

3.2. Base de datos

(El diseño de la base de datos se basa en los conceptos teóricos tomados de [11, Capítulos 6 y 7] y de los apuntes de la asignatura Base de Datos)

El primer paso para implementar una base de datos es su diseño. Para ello es importante seleccionar un modelo de representación de datos que estructure la información y las relaciones de distintos datos dentro de esta. En esta ocasión, se ha optado por el modelo Entidad-Relación.

El objetivo de este modelo es representar de forma gráfica los datos que se desean almacenar en la base de datos y las relaciones entre ellos. Es importante definir los distintos componentes del modelo:

- Entidad: Objeto del que se desea almacenar la información.
- Atributo: Característica que describe a la entidad.

- Relación: Asociación entre entidades. Esta se define no solo por su nombre, sino también por un conjunto de propiedades: cardinalidad, opcionalidad y grado.
  - Cardinalidad: número de elementos de una entidad que pueden asociarse con un elemento de otra entidad. Las más habituales son uno a uno $(1:1)$, uno a muchos $(1:N)$ y muchos a muchos $(N:M)$.
  - Opcionalidad: especifica si la participación de una entidad en una relación es obligatoria o no.
  - Grado: número de entidades que intervienen en una relación.

Identifiquemos sobre la información que tenemos cada uno de los elementos descritos:

- La entidad ***Estudiantes*** contiene a los alumnos matriculados en las distintas titulaciones bajo estudio. Sus atributos son: *id_est* (clave primaria), *nombre*, *apellido1*, *apellido2* e *id_grado*.
- La entidad ***Profesores*** almacena a los profesores que imparten las asignaturas. Sus atributos son: *id_prof* (clave primaria) y *nombre_prof*.
- La entidad ***Grados*** recoge la información de cada titulación. Sus atributos son: *id_grado* (clave primaria) y *nombre_grado*.
- La entidad ***Asignaturas*** almacena la información correspondiente a cada asignatura impartida en cada uno de los grados. Sus atributos son: *id_asig* (clave primaria), *nombre_asig* y *cuatri_asig*.
- La entidad ***Horario_nuevo*** recoge los días y las horas a las que se imparte cada asignatura. Sus atributos son: *dia* (clave primaria) y *hora* (clave primaria).

Una vez que tenemos identificadas las entidades, y sus correspondientes atributos, definimos las relaciones entre ellas:

Relación	Entidades	Cardinalidad	Opcionalidad	Grado
Alumno_de	Estudiantes - Grados	N:1	Obligatoria (Estudiante)	Binaria
Cursa	Estudiantes - Asignaturas	N:M	Obligatoria	Binaria
Profesor_de	Profesores - Asignaturas	N:M	Obligatoria (Profesor)	Binaria
Tiene_horario	Asignaturas - Horario_nuevo	1:N	Obligatoria (Horario)	Binaria
Se_imparte_en	Grados - Asignaturas	N:M	Obligatoria (Asignatura)	Binaria
A_la_vez	Asignaturas - Asignaturas	N:M	Opcional	Unaria

Con todo esto construimos el diagrama modelo Entidad-Relación que podemos ver en Figura 3.1.

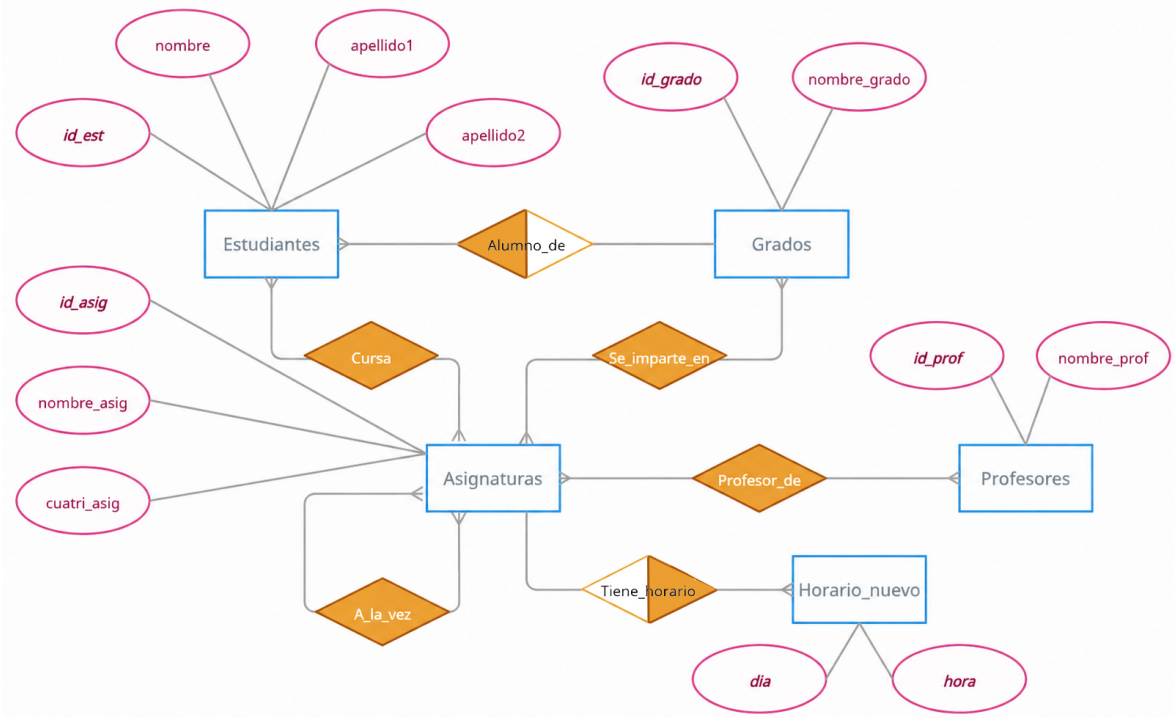


Figura 3.1: Diagrama modelo E-R

Tras definir el modelo Entidad-Relación, hay que transformar este al modelo relacional. Las entidades *Estudiantes*, *Profesores*, *Asignaturas*, *Horario_nuevo* y *Grados* se transforman directamente en tablas conservando sus claves primarias.

Las relaciones con cardinalidad $N : M$: *A_la_vez*, *Cursa*, *Profesor_de* y *Se_imparte_en* van a ser incorporadas a la base de datos creando nuevas tablas: $asig_doble$, rel_est_asig, rel_prof_asig y rel_grado_asig, respectivamente.

Para el caso de las relaciones *Alumno_de* y *Tiene_Horario* que tienen cardinal $N : 1$ y $1 : N$, respectivamente, optamos por implementarlas mediante la creación de un nuevo atributo en la tabla correspondiente al lado N de cada relación. Es decir, la relación *Alumno_de* se implementa mediante el atributo *id_grado* de la tabla $estudiantes$ y *Tiene_Horario* con *id_asig* en la tabla $horario_nuevo$.

La mayoría de las entidades del modelo tienen identificadores propios. Sin embargo, *Horario_nuevo* es una entidad débil pues su existencia depende de la entidad *Asignaturas*. Cada registro del horario tiene que estar asociado a una asignatura existente de manera obligatoria por lo que se ha definido una clave foránea sobre *id_asig*.

A continuación, se muestra la implementación física del modelo relacional. Las estructuras de las tablas que componen la base de datos se recogen en Tabla 3.1–Tabla 3.9.

Campo	Tipo de dato	Descripción	Clave primaria	Clave foránea
id_est	INT	Identificador único del estudiante	Si	No
nombre	VARCHAR(255)	Nombre del estudiante	No	No
apellido1	VARCHAR(255)	Primer apellido del estudiante	No	No
apellido2	VARCHAR(255)	Segundo apellido del estudiante	No	No
id_grado	INT	Indica el grado en el que está matriculado el estudiante	No	Si

Tabla 3.1: Estructura de la tabla `estudiantes`. Almacena los datos personales de los estudiantes registrados en el sistema.

Campo	Tipo de dato	Descripción	Clave primaria	Clave foránea
id_prof	INT	Identificador único del estudiante	Si	No
nombre_prof	VARCHAR(255)	Nombre del estudiante	No	No

Tabla 3.2: Estructura de la tabla `profesores`. Almacena el nombre compelto y el identificador de cada profesor.

Campo	Tipo de dato	Descripción	Clave primaria	Clave foránea
id_asig	INT	Identificador único de la asignatura	Si	No
nombre_asig	VARCHAR(255)	Nombre de la asignatura	No	No
cuatri_asig	INT	Cuatrimestre al que pertenece la asignatura según el plan educativo. A las optativas se les ha asignado 11 y 12, según corresponda	No	No

Tabla 3.3: Estructura de la tabla `asignaturas`. Registra las asignaturas de todas las titulaciones.

Campo	Tipo de dato	Descripción	Clave primaria	Clave foránea
id_grado	INT	Identificador único de la titulación	Si	No
nombre_grado	VARCHAR(255)	Nombre de la titulación	No	No

Tabla 3.4: Estructura de la tabla `grados`. Contiene las titulaciones consideradas en este trabajo.

Campo	Tipo de dato	Descripción	Clave primaria	Clave foránea
id_asig	INT	Identificador de la asignatura	Si	Si
dia	INT	Día de la semana	Si	No
hora	INT	Franja horaria del día. Se ha dividido cada mañana en 7 franjas horarias para la distribución de las asignaturas siendo la hora 1 de 8:15 a 9:05 y la 7 de 14:15 a 15:05	Si	No

Tabla 3.5: Estructura de la tabla `horario_nuevo`. Recoge cada asignatura junto con el día y la hora en la que se imparte. Esta tabla se crea con los datos obtenidos de resolver el problema de minimización P resuelto en el capítulo anterior de este trabajo.

Campo	Tipo de dato	Descripción	Clave primaria	Clave foránea
id_est	INT	Identificador único del estudiante	Si	Si
id_asig	INT	Identificador único de la asignatura	Si	Si

Tabla 3.6: Estructura de la tabla `rel_est_asig`. Relaciona estudiantes con asignaturas.

Campo	Tipo de dato	Descripción	Clave primaria	Clave foránea
id_prof	INT	Identificador único del profesor	Si	Si
id_asig	INT	Identificador único de la asignatura	Si	Si

Tabla 3.7: Estructura de la tabla `rel_prof_asig`. Relaciona profesores con asignaturas que imparten.

Campo	Tipo de dato	Descripción	Clave primaria	Clave foránea
id_grado	INT	Identificador único del grado	Si	Si
id_asig	INT	Identificador único de la asignatura	Si	Si

Tabla 3.8: Estructura de la tabla `rel_grado_asig`. Asocia asignaturas a titulaciones.

Campo	Tipo de dato	Descripción	Clave primaria	Clave foránea
id_asig1	INT	Identificador de la primera asignatura	Si	Si
id_asig2	INT	Identificador de la segunda asignatura	Si	Si

Tabla 3.9: Estructura de la tabla `asig_dobles`. Asocia asignaturas que se cursan simultáneamente.

La estructura general de la base de datos utilizada se muestra en Figura 3.2.

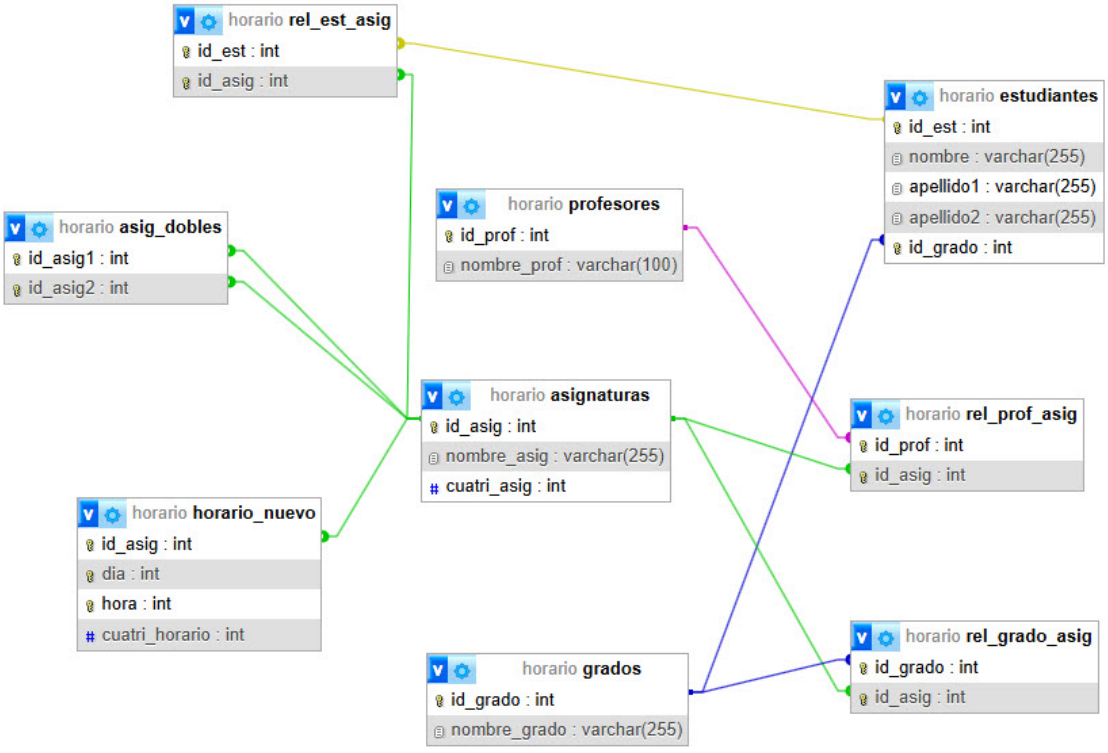


Figura 3.2: Estructura de la base de datos $horario$ con la que trabajamos

La tabla $asig_dobles$ (Tabla 3.9) almacena parejas de asignaturas que se imparten ya no solo de manera simultánea sino que es un único grupo en aula para dos titulaciones. Supongamos que ya hemos almacenado que la asignatura $id_asig = 1$ se imparte a la vez que aquella con $id_asig = 2$, es decir, $(1, 2)$ ya forma parte de los datos recogidos. Esta relación es simétrica, pues almacenar $(2, 1)$ no aporta información. Del mismo modo, tampoco tiene sentido permitir relacionar una asignatura consigo misma.

Para evitar la redundancia y la inconsistencia en la base de datos provocada por estas situaciones se ha implementado un disparador asociado a la tabla

$asig_dobles$. Este se ejecuta antes de insertar cada pareja de datos y verifica que las condiciones mencionadas no se cumplan.

Más allá de las restricciones específicas, una base de datos relacional debe garantizar la consistencia de las relaciones existentes entre las distintas tablas. Con este fin se utilizan las restricciones de integridad referencial que aseguran que los valores almacenados en las claves foráneas correspondan con valores registrados en las tablas referenciadas. De esta forma se evita la aparición de registros huérfanos.

También se han utilizado sentencias que implementan la actualización y borrado en cascada (`ON UPDATE CASCADE` y `ON DELETE CASCADE`) que mantienen de forma automatizada la coherencia de los datos.

Por último, comentar que hemos aplicado los principios de normalización para reducir la redundancia de datos. Todas las tablas cumplen la primera forma normal $1FN$, cada atributo almacena valores atómicos, es decir, indivisibles y sin repeticiones. Podemos comprobar que también se verifica la segunda forma normal $2FN$ que exige que todos los atributos que no forman parte de la clave primaria dependan de la clave completa y no de parte de ella. Esta se cumple de manera trivial pues en nuestro caso, todas las claves primarias tienen o un solo atributo o están formadas por todos los atributos de la tabla. Finalmente, es necesario verificar el cumplimiento de la tercera forma normal $3FN$. Esta forma exige que ningún atributo dependa de otro que no pertenezca a la clave primaria. En nuestro caso, la información de cada estudiante, profesor, asignatura y grado solo la determina su correspondiente id y, en el resto, todos los elementos pertenecen a la clave primaria.

3.3. Interfaces

Se han desarrollado dos páginas web mediante el uso de las siguientes herramientas: PHP (comunicación con la base de datos), $HTML$ (generación dinámica del contenido), $JavaScript$ (interacción con el usuario) y CSS (definición del aspecto visual).

La primera interfaz creada simula el Portal de Servicios (Figura 3.3), disponible para los estudiantes de la Uex. Genera los ficheros necesarios para poder ejecutar el código de $RStudio$. Está compuesta por tres archivos de código: $index.php$, $index_script.js$ y $index_style.css$ que separan la interacción con la base de datos, las estructuras dinámicas y el diseño gráfico de la página. Esta organización permite el desarrollo del código, su mantenimiento y la localización de errores.

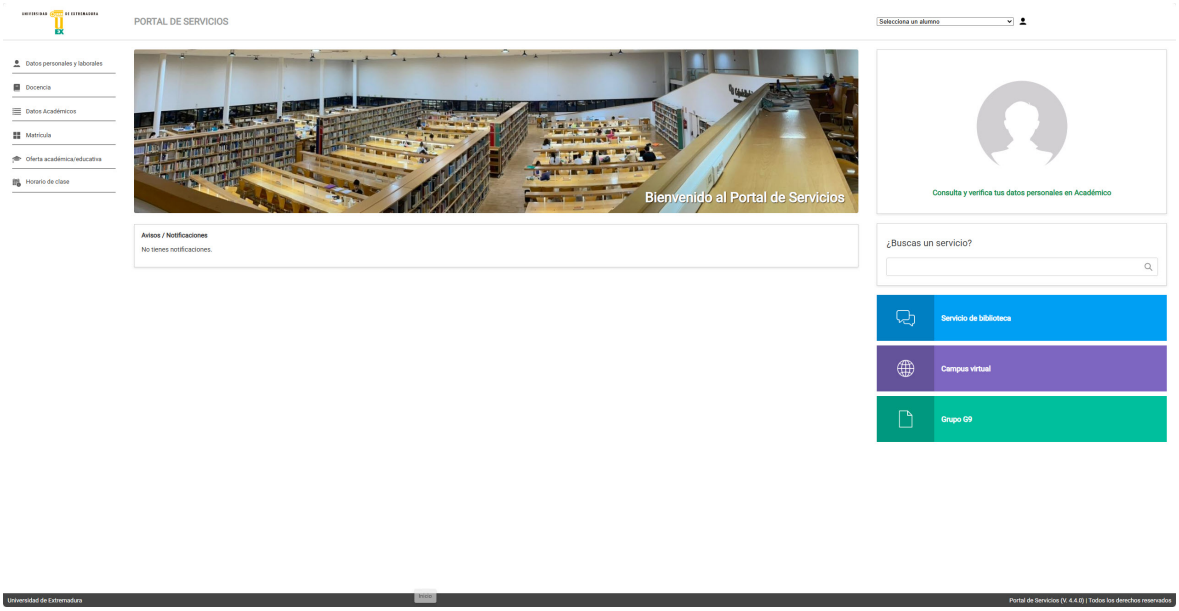


Figura 3.3: Recreación del Portal de Servicios

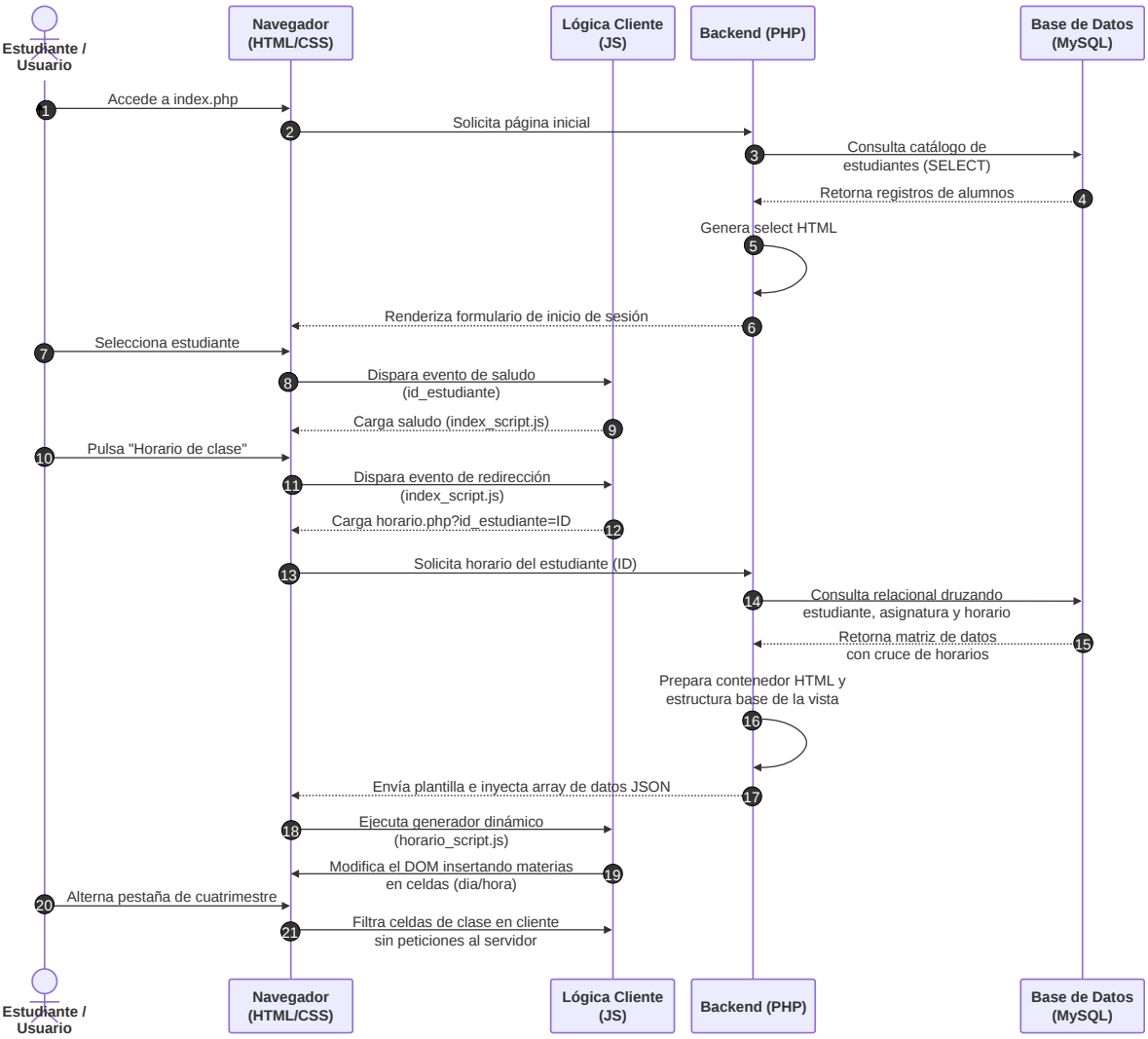


Figura 3.4: Diagrama de secuencia del proceso de consulta del horario personal

La segunda muestra el horario personal por cuatrimestres del estudiante seleccionado en la página previa. También está compuesta por tres archivos de código: $horario.php$, $horario_script.js$ y $horario_style.css$. El acceso a esta interfaz lo encontramos en el menú izquierdo de la pantalla, $Horario\ de\ clase$.

Antes de comenzar a describir cada uno de los archivos mencionados, es útil analizar el flujo de interacción del sistema mediante un diagrama de secuencia. La Figura 3.4 muestra el proceso completo desde el acceso a la página principal (Portal de Servicios) hasta la consulta del horario.

3.3.1. $index.php$

Dividido en dos partes claramente diferenciables: php encargada de la interacción con la base de datos y $html$ de la estructura.

La primera parte realizada con php comienza realizando la conexión a la base de datos. Es fundamental para poder acceder a los datos de esta, tanto para la consulta como para insertar nuevos.

Tras la conexión inicial se generan las consultas que nos permiten obtener los ficheros de datos que necesitamos para poder ejecutar los scripts de $RStudio$. Aunque es cierto que las consultas realizadas se pueden generar de forma más agrupada y crear menor cantidad de ficheros con mayor número de columnas, esto incrementa de manera exponencial el coste computacional y la dificultad a la hora de resolver el problema de programación entera, que no debemos olvidar que es nuestro objetivo principal.

Se crea una nueva tabla en la base de datos que almacena la solución del problema de optimización. En caso de que la tabla exista, se elimina para crearla de nuevo vacía antes de importar los datos.

La segunda parte, realizada con $html$, está encargada de la estructura de la página. Como podemos ver en Figura 3.3 está compuesta de tres partes: el encabezado, el cuerpo y el pie. En el encabezado de la página, a diferencia de la página real del Portal de Servicios, encontramos un $select$. Aquí escogemos un estudiante con la idea de simular el inicio de sesión desde su cuenta y así acceder a su horario personal. El identificador del estudiante seleccionado se envía a $horario.php$, donde se mostrará su horario.

En el cuerpo encontramos tres secciones en donde podemos destacar dentro del menú izquierdo la creación de un nuevo apartado «Horario de clase» el cual está activo y nos lleva al horario personal del alumno seleccionado, la segunda página creada.

(Código completo en Apéndice K)

3.3.2. *index_script.js*

Este código implementa el comportamiento dinámico de la página principal. Entre sus funciones se encuentran la carga de la información de los estudiantes, la actualización de los elementos (como puede ser la personalización del saludo que vemos en Figura 3.5) y la navegación hacia la página de consulta de horarios. También, se encarga de transmitir la información necesaria para que el horario correspondiente al estudiante pueda ser mostrado.

(Código completo en Apéndice L)

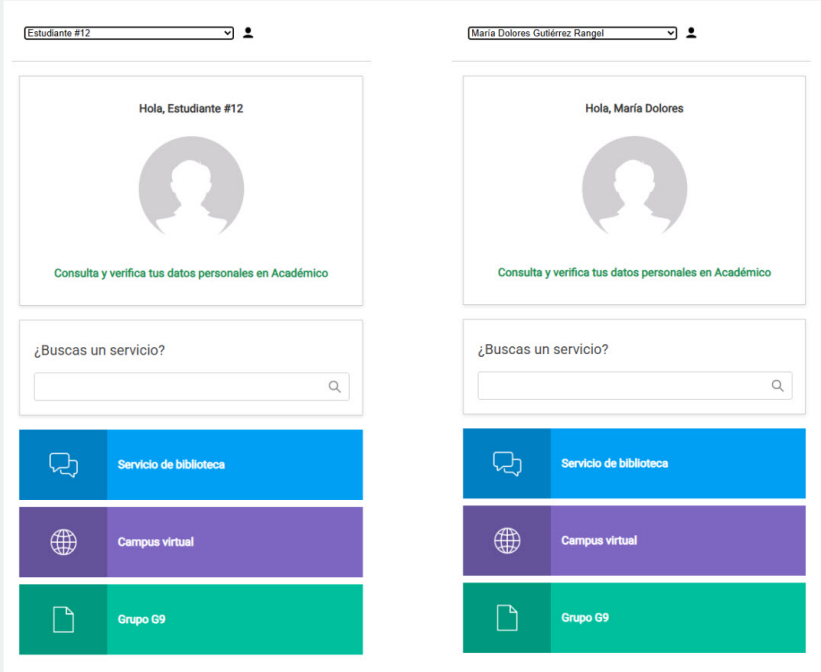


Figura 3.5: Ejemplos de distintas opciones según la información proporcionada por el estudiante

3.3.3. *horario.php*

Al igual que en el caso de *index.php*, tenemos dos partes claras, la que interactúa con la base de datos escrita en *php* y la que crea la estructura de la página web utilizando *html*. Realizamos las consultas necesarias para obtener las asignaturas del estudiante y las del grado en cuestión. Volvemos a crear ciertos elementos como el encabezado, el menú izquierdo y el pie de la página con los que ya contaba la página anterior con la idea de que mantenga la forma y dar sensación de continuidad y armonía al usuario.

(Código completo en Apéndice N)

3.3.4. $horario_script.js$

El código creado en $JavaScript$ es algo más complejo en esta ocasión que en el caso de $index_script.js$ que hemos comentado previamente.

La función principal genera dinámicamente la estructura base del horario y rellena las celdas con la información recibida de la base de datos (véase Figura 3.6). Se permite al usuario seleccionar entre los distintos cuatrimestres para visualizar su correspondiente horario personalizado. También encontramos la opción que muestra el horario de todos los semestres que forman la titulación al completo. En función de la opción tomada, la página se actualiza de forma automática construyendo los horarios necesarios.

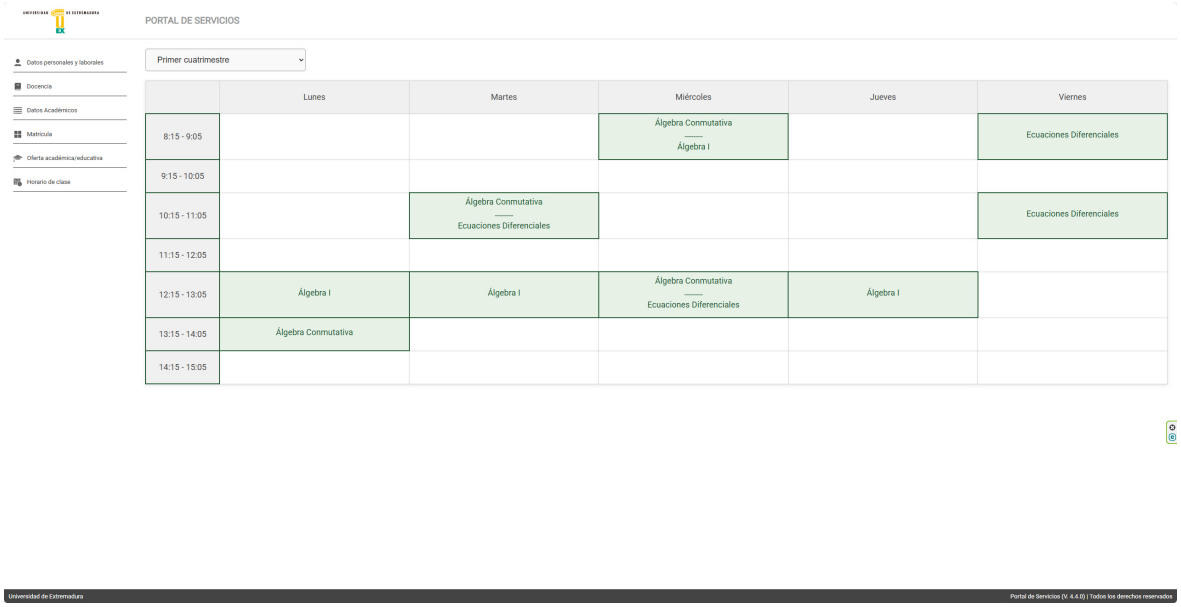


Figura 3.6: Muestra de la apariencia de la segunda página creada donde se puede observar el horario del primer cuatrimestre para un alumno concreto

(Código completo en Apéndice Ñ)

3.3.5. $index_style.css$ y $horario_style.css$

Estos dos códigos escritos con CSS se encargan de definir el aspecto visual de ambas páginas. Garantizan la apariencia homogénea entre ambas páginas manteniendo el estilo de los elementos comunes y personalizando aquellos distintos.

(Código completo en Apéndice M y Apéndice O)

Finalmente, se ha desarrollado un manual de despliegue en el que el lector

puede encontrar el procedimiento necesario para reproducir el entorno de ejecución utilizado en este trabajo. En Apéndice P se describen los requisitos de software, la configuración del sistema, la importación de la base de datos y los pasos necesarios para ejecutar el método heurístico seleccionado.

Conclusión

Este trabajo ha permitido consolidar y ampliar los conocimientos adquiridos a lo largo de la realización del grado en Matemáticas.

La optimización de horarios es un problema común en muchos ámbitos que presenta una gran complejidad al tratarse de un problema NP-completo, lo que justifica el empleo de métodos heurísticos.

El problema planteado ha permitido ampliar los conocimientos de investigación operativa mediante la programación entera, modelizando este ejemplo. Se han analizado y comparado diferentes modelos exactos y heurísticos obteniendo que el enfriamiento simulado es el método que ofrece mejores resultados en este problema.

Este trabajo no se ha limitado al ámbito de la optimización sino que se ha explorado la creación de una base de datos que gestione la información recabada. Esta estructura ha facilitado en gran medida la organización de los datos necesarios para la ejecución de los algoritmos y la posterior muestra de resultados.

Además, el diseño de una página web conectada a dicha base ha hecho posible trasladar los resultados obtenidos a cada alumno, mostrando de manera clara su aplicación directa.

Como posibles futuras mejoras, el modelo podría ampliarse incorporando restricciones nuevas, como la disponibilidad de aulas y el aforo de estas. Del mismo modo, la interacción con la base de datos podría extenderse y ofrecer los horarios personalizados por profesor junto a información importante como el despacho o el horario de tutoría.

Bibliografía

[1] Juan Manuel Izar Landeta. *Modelos matemáticos para la toma de decisiones*. IMCP (Instituto Mexicano de Contadores Públicos), 2014.

[2] Wolsey, L. A. & Nemhauser, G. L. *Integer and combinatorial optimization*. Wiley, 1999.

[3] Hillier, F. S., & Lieberman, G. J. *Introduction to operations research*. McGraw-Hill, 2005.

[4] Mokhtar S. Bazaraa, John J. Jarvis, Hanif D. Sherali. *Linear Programming and Network Flows*. John Wiley & Sons, 2010.

[5] Michel Gendreau, Jean-Yves Potvin (eds.) *Handbook of Metaheuristics*. Second Edition, Springer, 2010.

[6] El-Ghazali Talbi. *Metaheuristics: From Design to Implementation*. John Wiley & Sons, 2009.

[7] Sipser, M. *Introduction to the Theory of Computation*. Cengage Learning, 2013.

[8] Scrucca, L. *GA: A Package for Genetic Algorithms in R*. CRAN Repository. Disponible en: https://cran.r-project.org/web/packages/GA/GA.pdf.

[9] Xiang, Y., Gubian, S., Suomela, B. y Hoeng, J. *GenSA: Generalized Simulated Annealing for Global Optimization*. CRAN Repository. https://cran.r-project.org/web/packages/GenSA/GenSA.pdf

[10] Domijan, K. *tabuSearch: Tabu Search Algorithm for Binary Configurations*. CRAN Repository. https://cran.r-project.org/web/packages/tabuSearch/tabuSearch.pdf

[11] Silberschatz, A., Korth, H. F., y Sudarshan, S. *Fundamentos de Bases de Datos (5.ª ed.)*. McGraw-Hill, 2006.

[12] Garey, M. R., y Johnson, D. S. *Computers and Intractability: A Guide to the Theory of NP-Completeness*. W. H. Freeman and Company, 1979.

Apéndice P

Manual de despliegue

Este proyecto ha sido desarrollado y probado en un entorno Windows utilizando WAMPServer para la ejecución de la página web y la implementación de la base de datos, y RStudio para el desarrollo de los métodos heurísticos.

En este apéndice se van a aclarar aspectos fundamentales con el objetivo de facilitar al lector ejecutar los códigos utilizados y poder replicar el entorno utilizado.

En primer lugar, la localización de los archivos que constituyen este proyecto debe tener una forma determinada, véase Figura P.1.

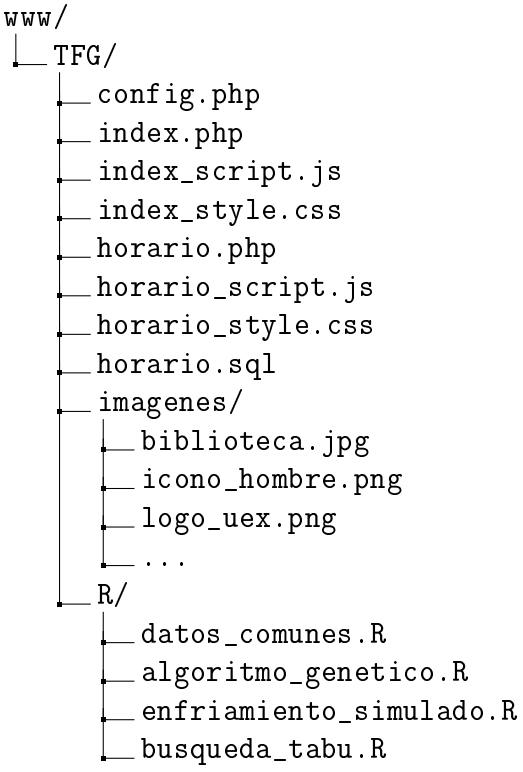


Figura P.1: Estructura de directorios del proyecto

En primer lugar es necesarioes necesario disponer del siguiente software que es gratuito y de facil instalación: $WAMPServer$ (versión 3.3.2 o superior), $RStudio$ (versión 4.4.2 o superior) y un navegador web. Se recomienda, además, disponer de un editor de código como puede ser $Visual\ Studio\ Code$.

Se desglosa a continuación el paso a paso para replicar el entorno necesario para la ejecución completa del trabajo:

- Instalación base de datos:
    1. Iniciar $WAMPServer$
    2. Acceder a localhost: `http://localhost/phpmyadmin`
    3. Crear una nueva base de datos denominada *horario*
    4. Importar archivo $horario.sql$

  Tras ejecutar la importación la base de datos está lista para ser utilizada.
- Configuración: Los parámetros dependientes del entorno se encuentran localizados en único fichero $config.php$. Aquí se configuran las credenciales de acceso a la base de datos (Servidor MySQL, nombre de la base de datos, usuario y contraseña) y se define el directorio de la carpeta *R* utilizada para almacenar los ficheros que utiliza posteriormente $RStudio$.

  Cualquier modificación en el entorno de ejecución solo necesita ser realizada en este archivo sin necesidad de alterar el resto de códigos.

  (Código completo en Apéndice Q)
- Generar archivos: Los códigos de $RStudio$ que optimizan el horario necesitan una serie de archivos que se generan de forma dinámica al iniciar la página web diseñada por lo que necesitamos entrar por primera vez antes de optimizar el horario: `http://localhost/TFG`
- Optimizar horario:
    1. Abrir $RStudio$ y en la pestaña *Session* fijar el directorio de trabajo. Hay que seleccionar el directorio de la carpeta *R* de Figura P.1.
    2. Abrir $datos_comunes.R$ y $enfriamiento_simulado.R$
    3. Desde $datos_comunes.R$ ejecutar las primeras líneas para instalar las librerías necesarias. Este paso no es siempre necesario, dependerá de la versión de RStudio disponible.
    4. Ejecución del código completo de $enfriamiento_simulado.R$.

  Al finalizar la ejecución se crea un fichero $solucion_horario.csv$ que contiene el nuevo horario y será utilizado posteriormente por la página web.

- Consulta de horario:
    1. Acceder al servidor local: http://localhost/TFG
    2. Seleccionar, en la parte superior derecha de la pantalla, el alumno del que se desea consultar el horario
    3. Acceder al apartado *Horario de clase* disponible en el menú de la parte izquierda
    4. Seleccionar el horario que se desea consultar (primer cuatrimestre, segundo cuatrimestre u horario completo de la titulación)

  Para consultar el horario de otro estudiante se repite esta última parte.